

Git and GitHub

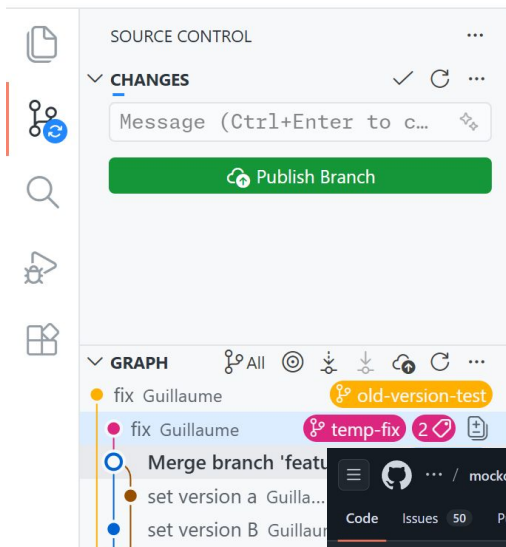


git

GitHub

Tools

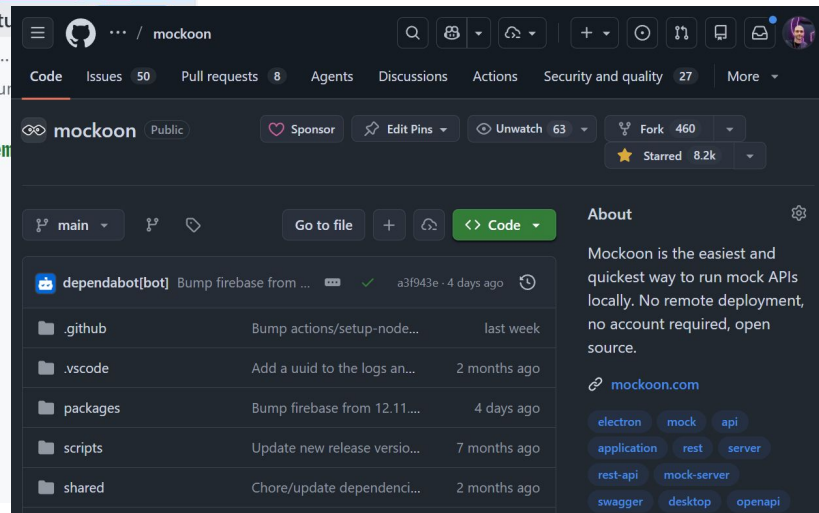
- [Visual Studio Code](#)
- [Git](#) / Git bash
- [github.com](#)



```
gmonn git-course old-version-test # git show v1.0.0
commit 6317ae58557a2306b09b109c875becb9cf3c36b4 (tag: v2.0.0, tag: v1.0.0, tem
Author: Guillaume <guillaume@1kb.software>
Date: Thu Apr 30 20:26:23 2026 +0200

    fix

diff --git a/fix.txt b/fix.txt
new file mode 100644
index 0000000..0e0df73
--- /dev/null
+++ b/fix.txt
@@ -0,0 +1 @@
gmonn git-course master # git tag
v1.0.0
v2.0.0
```



Resources

- [Git tutorials](#)
- [github.com tutorial](#)
- [GitKraken tutorials](#)
- [Interactive tutorial](#)
- [Git HowTo](#)

Intro

What is git?

Git is a distributed **version control system**. It allows multiple people to work simultaneously on a project, track changes, and revert to previous states if necessary.

Key Features:

- **Distributed** nature: Each developer has a complete copy of the repository.
- Change **history**: Detailed tracking of every modification made.
- Facilitated **collaboration**: Simplified management of teamwork.



What is GitHub?

GitHub is a web **platform** that **uses Git** for version control. It offers a complete **collaborative environment** for developers.

Key Features:

- Repository **hosting**: Stores your Git projects online.
- **Project management** tools: Management of issues, pull requests, etc.
- **Code review**: Facilitates discussions and peer code reviews.

And more: CI/CD, artifact/release hosting, etc.

 Important: **Git ≠ GitHub**. Git is the technical tool, whereas GitHub is the hosting service and the social/project management interface.

Why use version control?

Collaboration: **Multiple developers can work** on the same project without interfering with each other.

History: Ability to **track changes over time** and **revert** if necessary.

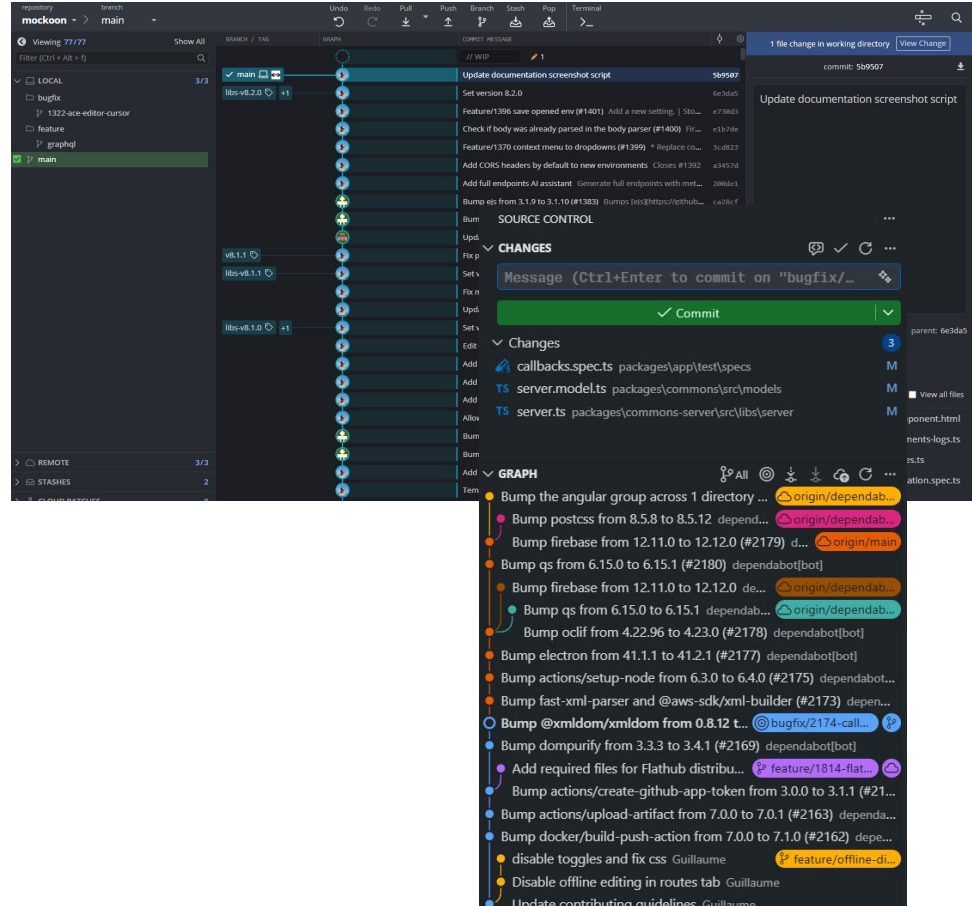
Branching: **Experimenting** with new features or fixes **without affecting the main codebase**.

Backup: Code is stored in remote repositories, **protecting against local data loss**.

Command-line vs GUI

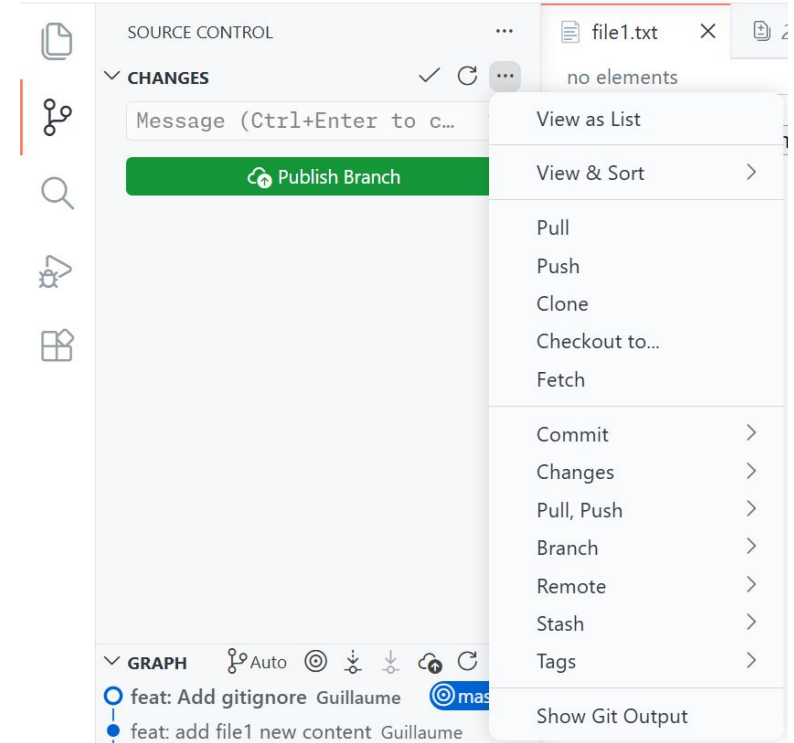
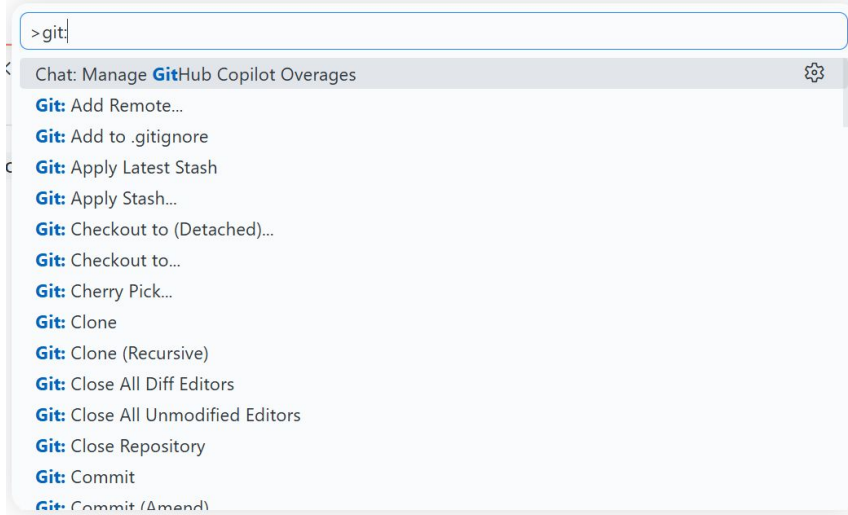
Git is a **command-line tool** including numerous commands such as `git commit`, `git push`, `git pull`, `git add`, etc. Everything can be done via the terminal, but it is sometimes difficult to get a big-picture view.

Graphical User Interfaces (GUIs) can help in understanding the state of the code and often facilitate advanced operations.

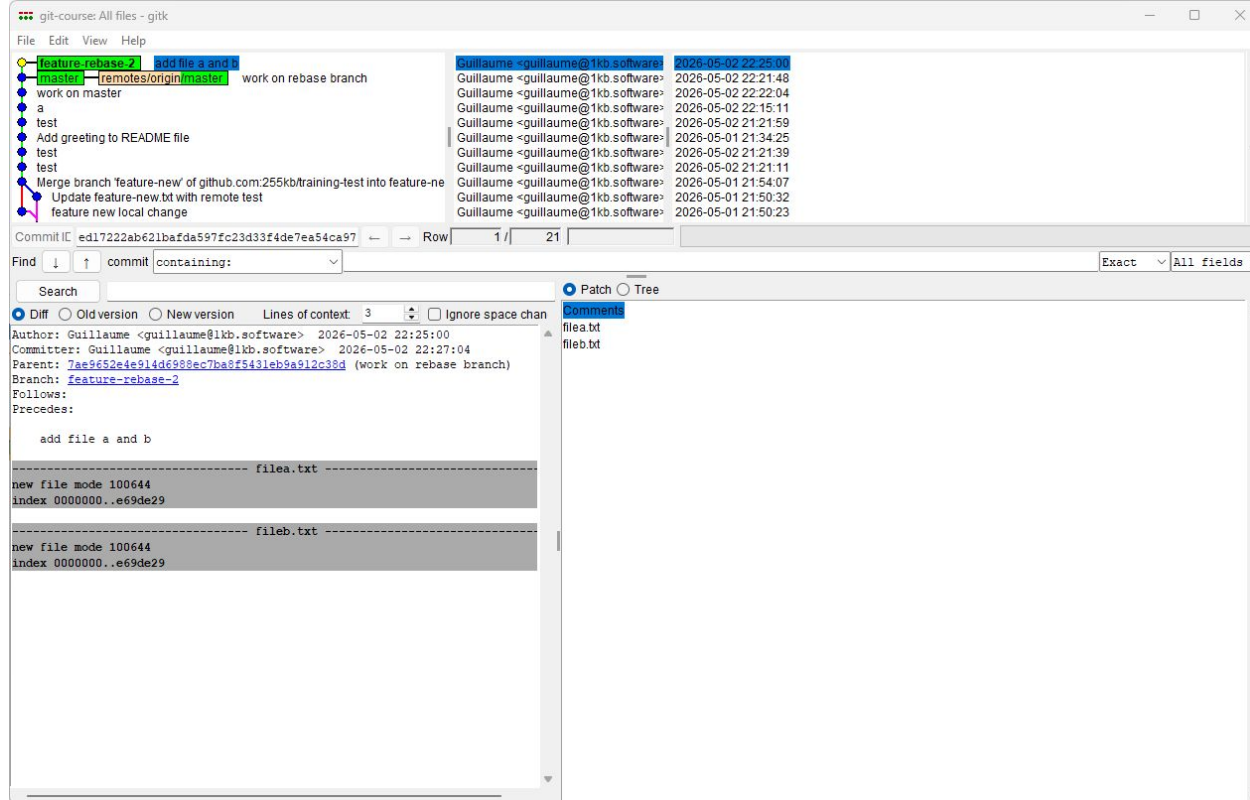


UI example: VSCode

All the commands we are going to use have their equivalent in the VSCode menus, under the Source Control section,  or via the command palette.



UI example: gitk (a git command)



What is a repository?

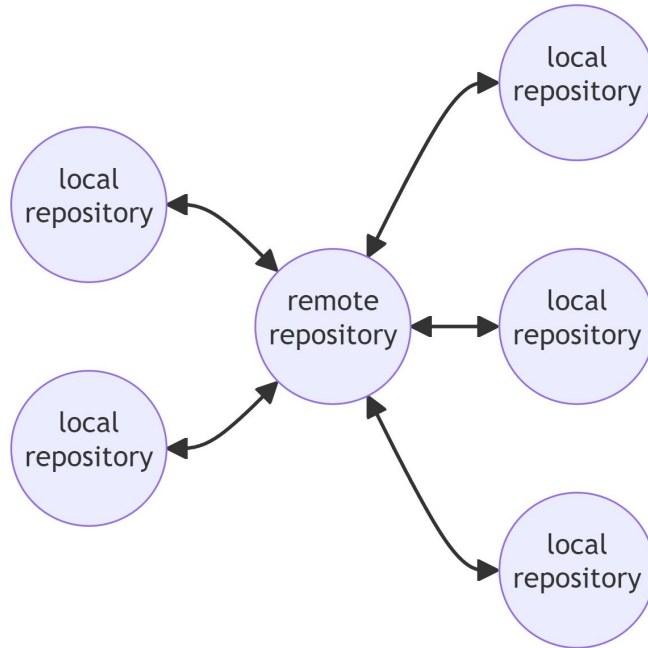
A **repository** (often called "repo") is a **location where project files are stored**. Repositories contain the entire source code of a project, including all versions and the history of that project.

- With Git, this translates into a single **folder** containing your **project files** and a hidden **.git folder** containing the local Git repository (a sort of database).
- A **local repository** is stored on a **local machine** (your computer).
- A **remote repository** is **hosted** on a **server** or a platform like GitHub, GitLab, or Bitbucket. It also contains the project files and the Git history.



Git is distributed

Git is **decentralized** and **distributed**. Each repository is a **copy**, and there is no real single source of truth (although the remote repository, or "remote", can be treated as such).



Exercise 1: setup

Objectives: Install Git, configure user identity, and initialize a first local repository.

1. Install Git from the official website git-scm.com
2. Open Git Bash (or your default terminal).
3. Configure your Git identity with the following commands:
 - **git config --global user.name "Votre Nom"**
 - **git config --global user.email "votre@email.com"**
4.  For now, we are working locally, but when we use GitHub, this email will potentially be public and included in every commit! You must either change the configuration when publishing to a public repo, or use an email that can be public.
5. Verify that the configuration has been correctly saved by listing the parameters: **git config --list** (you may need to exit by pressing the "q" key).

Exercise 1: setup

5. Create a new folder for your project and open it with Visual Studio Code (install it if you haven't already done so) either via the file explorer or with the command code .
6. In the VS Code terminal (or your external terminal positioned in the folder), run the command: **git init**
7. We don't see it in VSCode, but a **.git** folder has been created. You can use the **cd** and **ls** commands to explore this folder, or your operating system's file explorer.

```
./ ../ config description FETCH_HEAD HEAD hooks/ info/ objects/ refs/
```

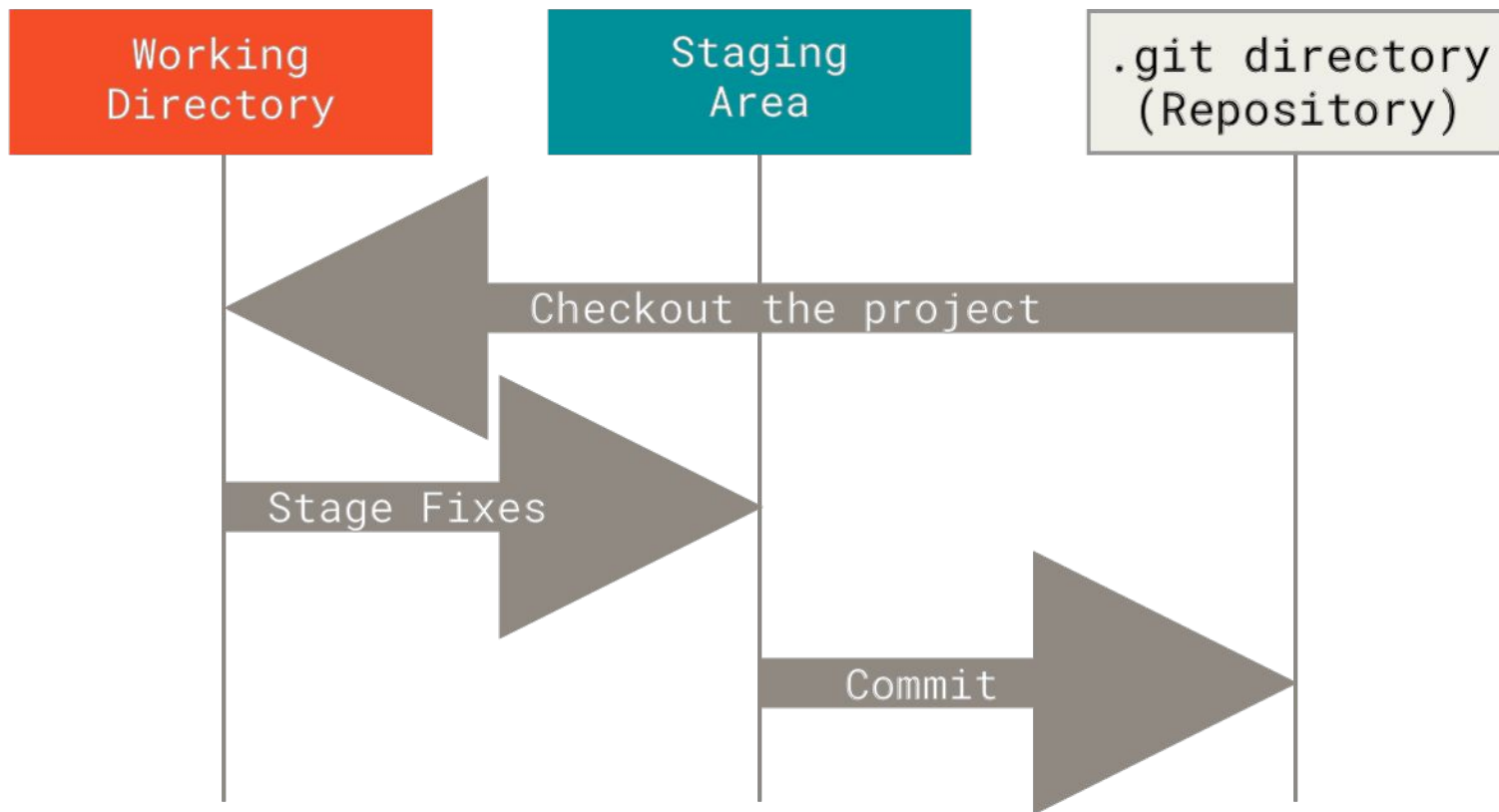
8. Git is now ready to track changes in your project.

Working directory, staging and commits

Git's three zones

Git manages your project through three distinct areas:

- **Working Directory:** The files as they appear on your computer. This is where you make your modifications.
- **Staging Area:** An intermediate area where you list the changes you want to include in your next save (commit).
- **Local Repository:** The database where Git permanently stores your version history (the **.git folder**).



Why a staging zone?

The staging area allows you to **prepare your commits** with precision.

You can modify ten files but decide to "commit" only two of them to keep a clear and thematic history (e.g., one commit for a bug fix, another for a new feature).

Files lifecycle

Each file in your project goes through different states:

- **Untracked:** The file is present in the folder, but Git does not know about it yet.
- **Modified:** The file is tracked by Git and has undergone changes, but is not yet ready to be saved.
- **Staged:** The file is marked as part of the next backup batch.
- **Committed:** The file is securely recorded in the history.

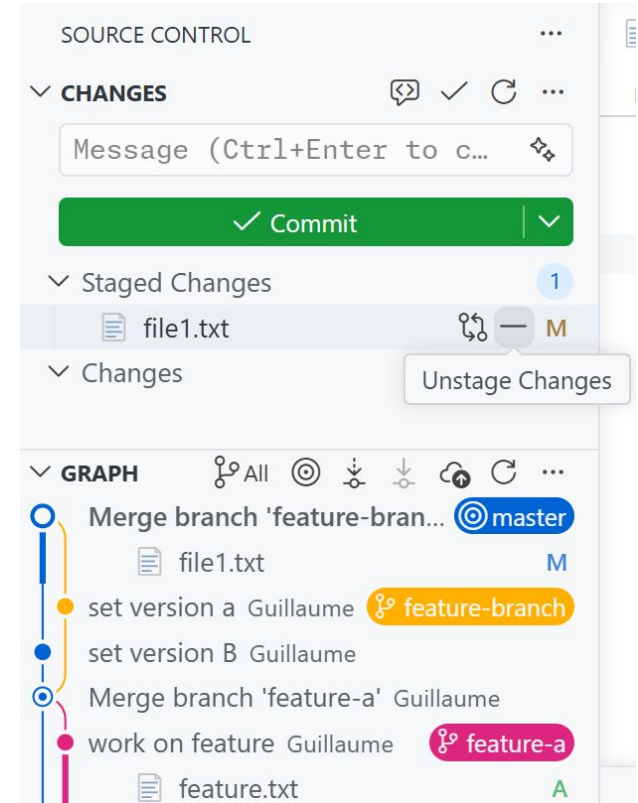
Basic commands

- **git status** : Allows you to see the state of the files (which areas they occupy).
- **git add <file_name>** : Moves a file from the working directory to the staging area.
- **git commit -m "Validation message"** : Saves the files from the staging area to the local repository.

VSCode: status, add and commit

The VSCode interface allows you to perform most tasks that are done with commands.

Here, it is possible in a single place to see the repository status, move changes into the staging area, or commit while adding a message.

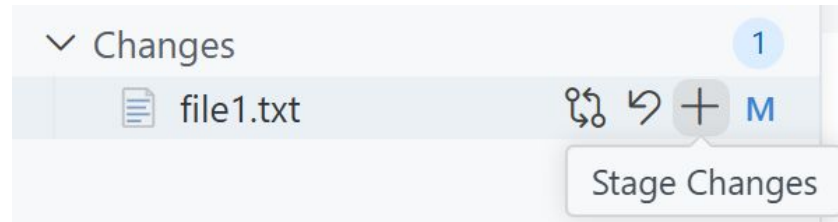


Git add

The **git add** command allows you to move changes from the working directory to the staging area. This is the preparation step before the final save.

The Different Variants:

- **git add filename** : Stage a specific file.
- **git add .** : Stage all project modifications (modified files, new files, and deletions) from the root directory (or a specific folder).



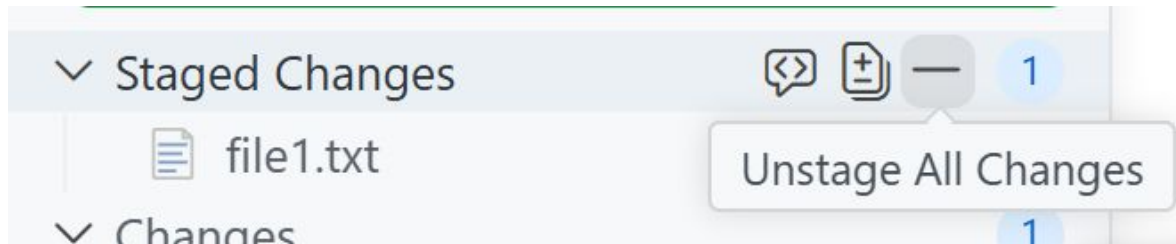
Git “remove”

If you accidentally added a file to the staging area, you can remove it while keeping it in the working directory:

- **git rm --cached -r .** (or **git rm --cached <filename>**)

--cached: Targets only the staging index.

-r: Allows recursive removal (necessary if the target is a folder).



Git status

The git status command is your permanent dashboard. It allows you to know precisely where your files stand in relation to the different Git areas.

Information Displayed:

- **Changes to be committed:** Files present in the staging area, ready for the next commit.
- **Changes not staged for commit:** Files modified in your working directory, but not yet added to the staging area.
- **Untracked files:** Recently created files whose history Git is not yet tracking.

Correspondence in VS Code, under the Source Control tab (shortcut Ctrl+Shift+G):

- **U (Untracked):** Untracked file.
- **M (Modified):** Modified file.
- **A (Added):** File added to the staging area.
- **Staged Changes:** Visual equivalent of the staging area.

Git status

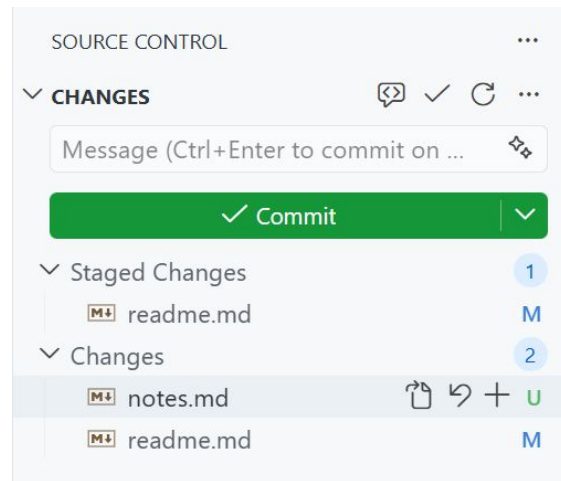
```
gmonn > git-course master # ? ?1 ? +1 > ? git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   readme.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   readme.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.md
```



Exercise 2: staging zone

Objectives: Manipulate files between the working directory and the index, and learn how to undo staging.

1. Create two files: **file1.txt** and **file2.txt** with any content.
2. Verify the state with **git status** (the files should appear as "**Untracked**").
3. Add only **file1.txt** to staging with **git add**.
4. Verify again with **git status**.
5. Remove **file1.txt** from staging without deleting it from the disk by using **git rm --cached file1.txt**
6. Verify with **git status** that both files are back in the "**Untracked**" state.

What is a commit?

A commit is a **snapshot** of your repository at a specific moment in time. It represents a set of modifications or an update to the project files. Each commit has a **unique identifier called a SHA** (Secure Hash Algorithm) hash. This hexadecimal code allows you to precisely locate and reference each state of the project.

A commit also contains:

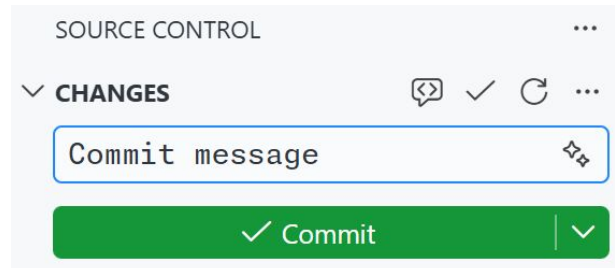
- A descriptive **message**.
- The **author's identity** (name and email).
- The **date** and **time** of creation.
- The **SHA** of the parent commit (the previous commit).



Save the changes: git commit

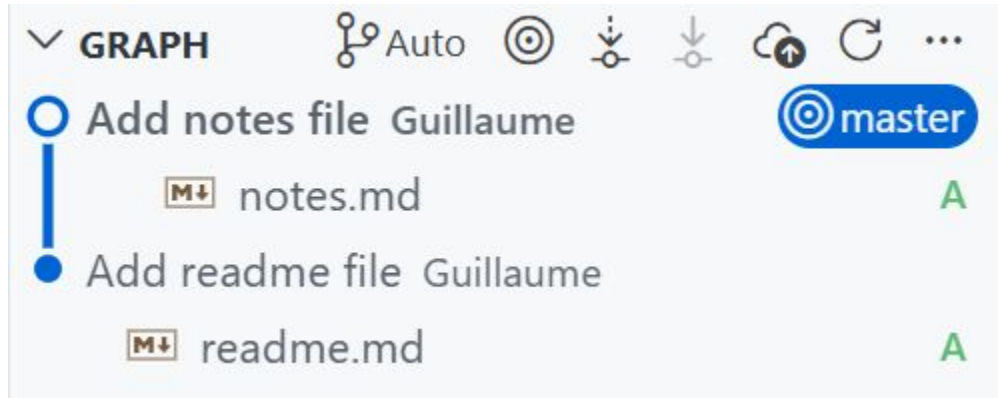
The git commit command validates the content of the **staging** area and **records** it permanently in your local history.

- **git commit -m "Message"**: This allows you to write the validation message directly in the command line. It is the preferred method for short, straightforward messages.
- **git commit**: If you run the command without the -m option, Git will automatically open your default text editor (such as Vim, Notepad++, or VS Code). The commit will only be created once you save the message file and close that editor.



Creating a commit

```
gmonn git-course master # ?1 git commit -m "Add notes file"
[master 8c3a659] Add notes file
1 file changed, 1 insertion(+)
create mode 100644 notes.md
```



Write good commit messages

Ideally, a commit message should describe what you have done, but not necessarily what the code does. Much like comments within the code, use the message to add any relevant information or the context of this change.

Projects may adopt different styles for writing commit messages. There are widely adopted conventions, such as **Conventional Commits**.

The Conventional Commits format generally follows this structure:

<type>[**optional scope**]: **<description>**
[body]
[footer]

Example:

fix: menu not closing correctly
Trigger menu closing in several uncovered cases...
Closes #1234

 [Conventional Commits](https://conventionalcommits.org)

History

View change history

To check the project's evolution and the content of your saves, several commands are essential:

Viewing History: **git log**

- **git log**: Displays the list of commits in reverse chronological order.
- **git log --oneline**: A simplified version displaying each commit on a single line (short SHA and message).
- **git log --parents**: Displays the parent commit identifier for each entry, allowing you to track the technical lineage of versions.

Inspecting a Specific Commit: **git show**

- **git show [SHA]**: Displays the complete metadata of a commit (author, date, message) as well as the exact line-by-line modifications (the "diff") introduced by that commit.

View change history

Git allows us to compare states with the command: **git diff**

- **git diff [SHA_1] [SHA_2]**: Compares two specific points in the history. This command shows the exact differences between two snapshots of the project.
- **git diff**: Without arguments, displays the current modifications in the working directory that have not yet been added to the staging area.

git log

```
● gmonn > git-course > master # > ✓ git log  
commit 8c3a659ca35684a3d2c44cec748bbff342aef607 (HEAD -> master)  
Author: Guillaume <guillaume@1kb.software>  
Date: Thu Apr 30 11:21:54 2026 +0200
```

Add notes file

```
commit 59b078c5bb0423781a7173d9b675aa10aaf6944f  
Author: Guillaume <guillaume@1kb.software>  
Date: Thu Apr 30 11:06:47 2026 +0200
```

Add readme file

git show [SHA]

```
gmonn  git-course  master #  git show 2f195bc9a06d7ea74c4c30b8241536955e239dd2
commit 2f195bc9a06d7ea74c4c30b8241536955e239dd2 (HEAD -> master)
Author: Guillaume <guillaume@1kb.software>
Date:   Thu Apr 30 11:15:43 2026 +0200
```

Add notes file

```
diff --git a/notes.md b/notes.md
new file mode 100644
index 0000000..30d74d2
--- /dev/null
+++ b/notes.md
@@ -0,0 +1 @@
+test
\ No newline at end of file
```

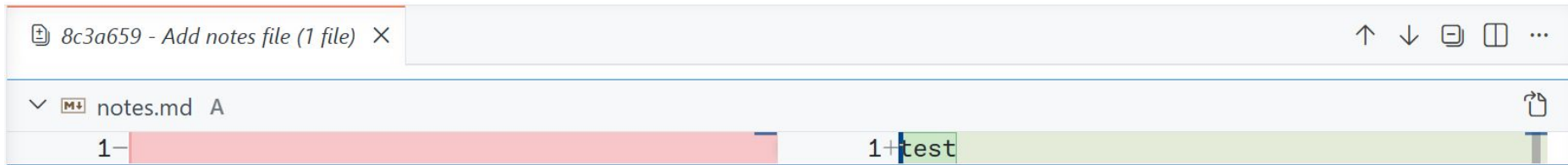
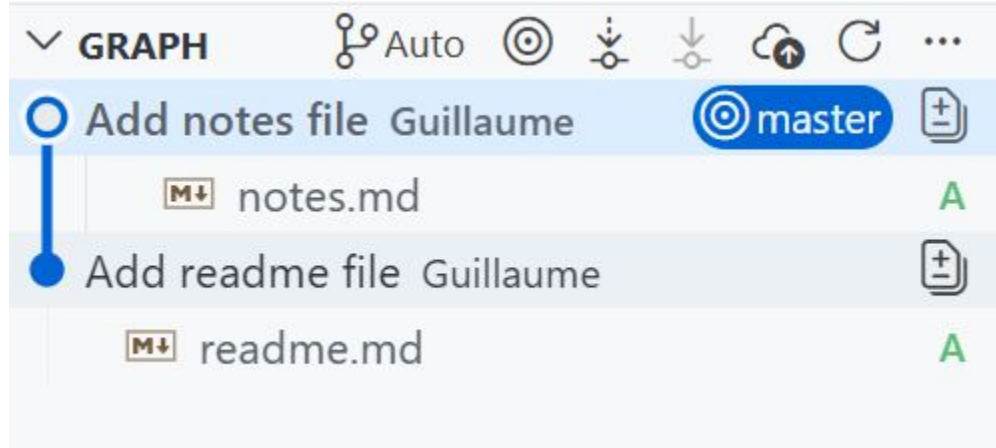
git diff [SHA_1] [SHA_2]

```
gmonn git-course master # git diff 8c3a659ca35684a3d2c44cec748bbff342aef607 59b078
diff --git a/notes.md b/notes.md
deleted file mode 100644
index 30d74d2..0000000
--- a/notes.md
+++ /dev/null
@@ -1,0 @@
-test
```

git diff (staging)

```
gmonn git-course master # ?2 git diff
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
diff --git a/file1.txt b/file1.txt
index 30d74d2..fc3eb2b 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,2 @@
-test
\ No newline at end of file
+test
+new content
\ No newline at end of file
```

VSCode's graph and diff



Exercise 3: commits and history

Objectives: Record versions, modify the project, and compare states.

1. [optional step] Several times, git will ask you for a message (commits, merge, etc.). It will open vim by default. To use VSCode as the default editor instead, run the following command: **git config --global core.editor "code --wait"**
2. Add both files (**file1.txt** and **file2.txt**) to the staging area using **git add**.
3. Perform a first commit with **git commit -m "feat: add base files"**.
4. Modify the content of **file1.txt** and save it.
5. Run **git diff** to see the added/deleted lines in the terminal.

Exercise 3: commits and history

6. Add the modification to the staging area and perform a second commit with an explanatory message.
7. Use **git log --oneline** to visualize your two backup steps.
8. Use **git show** followed by the SHA (the first few characters) of the last commit to view the details of the change.

.gitignore

.gitignore

The .gitignore file is a **text file** located at the root of the project that tells Git which **files or folders it should completely ignore**. This is important for several reasons:

- **Security:** To avoid uploading environment variables containing passwords or API keys (e.g., .env).
- **Size:** To prevent storing dependency folders (e.g., node_modules).
- **Cleanliness:** To avoid polluting the history with temporary or build files (e.g., .DS_Store, dist/, *.log).

 [Templates gitignore](#)

.gitignore

```
1  # Angular specific
2  /dist/
3  /out-tsc/
4  /tmp/
5  /coverage/
6  /e2e/test-output/
7  /.angular/
8  .angular/
9
10 # Node modules and dependency files
11 /node_modules/
12 /package-lock.json
13 /yarn.lock
14
15 # Environment files
16 /.env
17
18 # Angular CLI and build artefacts
19 /.angular-cli.json
20 /.ng/
21
22 # TypeScript cache
23 *.tsbuildinfo
24
```

Exercise 4: .gitignore

Objective: Create a .gitignore file and understand how it works.

1. Create a file named **secret.txt** and a folder named **logs/** containing a **debug.log** file inside it.
2. Run a **git status**: Git sees these files as "Untracked".
3. Create a file named **.gitignore** at the root of your project (the dot . at the beginning of the name is important!). Write these two lines inside the file: "**secret.txt**" and "**logs/**." Save the **.gitignore** file. The excluded files may now appear grayed out in the VSCode file explorer (depending on your theme).
4. Run **git status** again: the ignored files and folders should have disappeared from the list.
5. Create a commit containing the **.gitignore** file itself. This file is meant to be part of the codebase and shared with your team.

Stash

git stash

git stash allows you to **temporarily set aside** your current modifications without making a commit. This brings your working directory back to a "clean" state.

Key Commands

- **git stash / git stash push**: Saves your current modifications into a stack. You can name your stashes using -m "stash name" to easily manage and store multiple ones.
- **git stash pop**: Retrieves the most recent modifications from the stack, reapplies them to your working directory, and removes them from the stash list.
- **git stash list**: Displays a list of all the stashes you have set aside.
- **git stash apply**: Reapplies the modifications to your working directory but keeps them in the stash list as a backup safety measure.

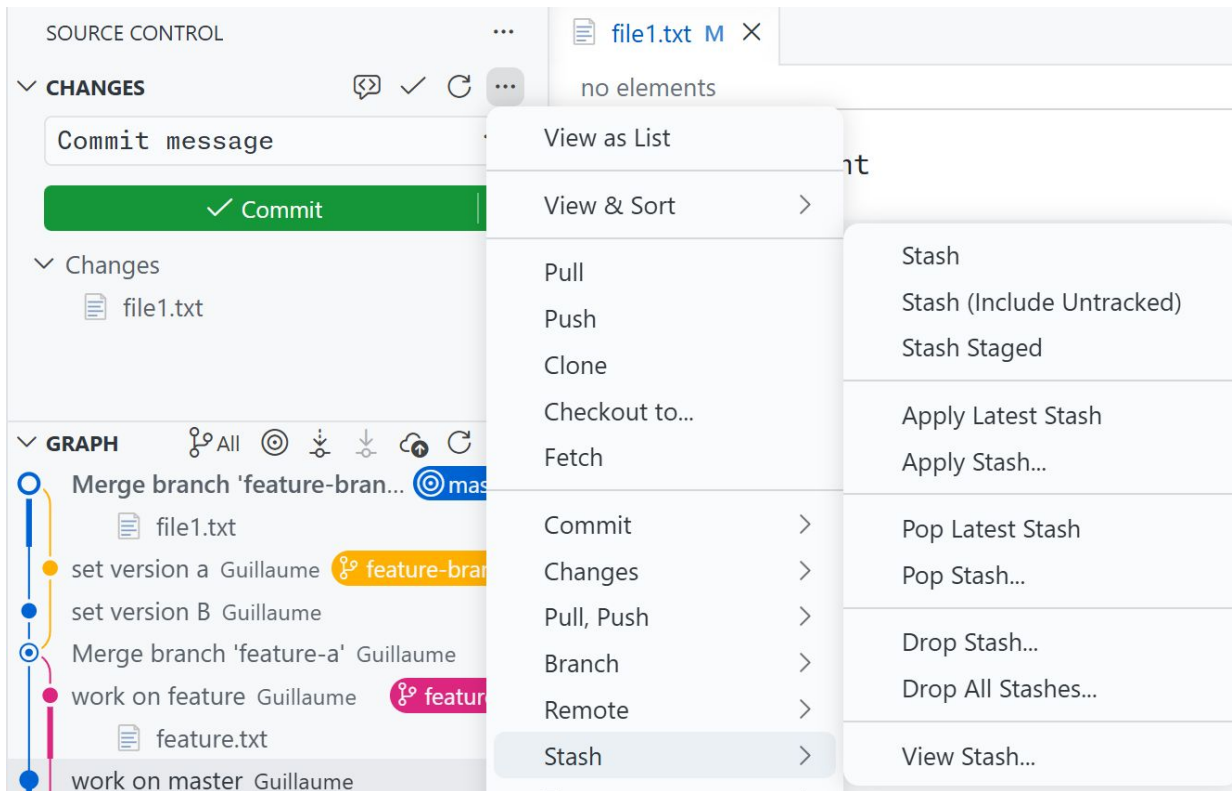
git stash

- gmonn git-course master # ~1 git stash push -m hello
Saved working directory and index state On master: hello

- gmonn git-course master # git stash apply
On branch master
Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 (use "git restore <file>..." to discard changes in working directory)
 modified: file1.txt

no changes added to commit (use "git add" and/or "git commit -a")

git stash



Exercise 5: git stash

Objective: Use stash commands to save and restore work in progress.

1. Modify the content of **file1.txt** by adding any content you like. Save the file.
2. Run the command **git stash**. The file returns to its original state (the last commit).
3. Verify with **git status**: "nothing to commit, working tree clean".
4. Use **git stash list** to view the stashes, then **git stash pop** to recover your work.
5. Verify that the modifications to **file1.txt** have returned.

Branches

Branches

A branch represents an independent line of development composed of one or more commits intended to be merged later.

Why Use Branches?

- **Isolate Work:** Develop a new feature without risking breaking the stable version (often named main or master).
- **Collaborate:** Each developer works on their own branch without interfering with anyone else's code.
- **Experiment:** Test an idea; if it doesn't work, you can simply delete the branch without impacting the rest of the project.

The **HEAD** pointer

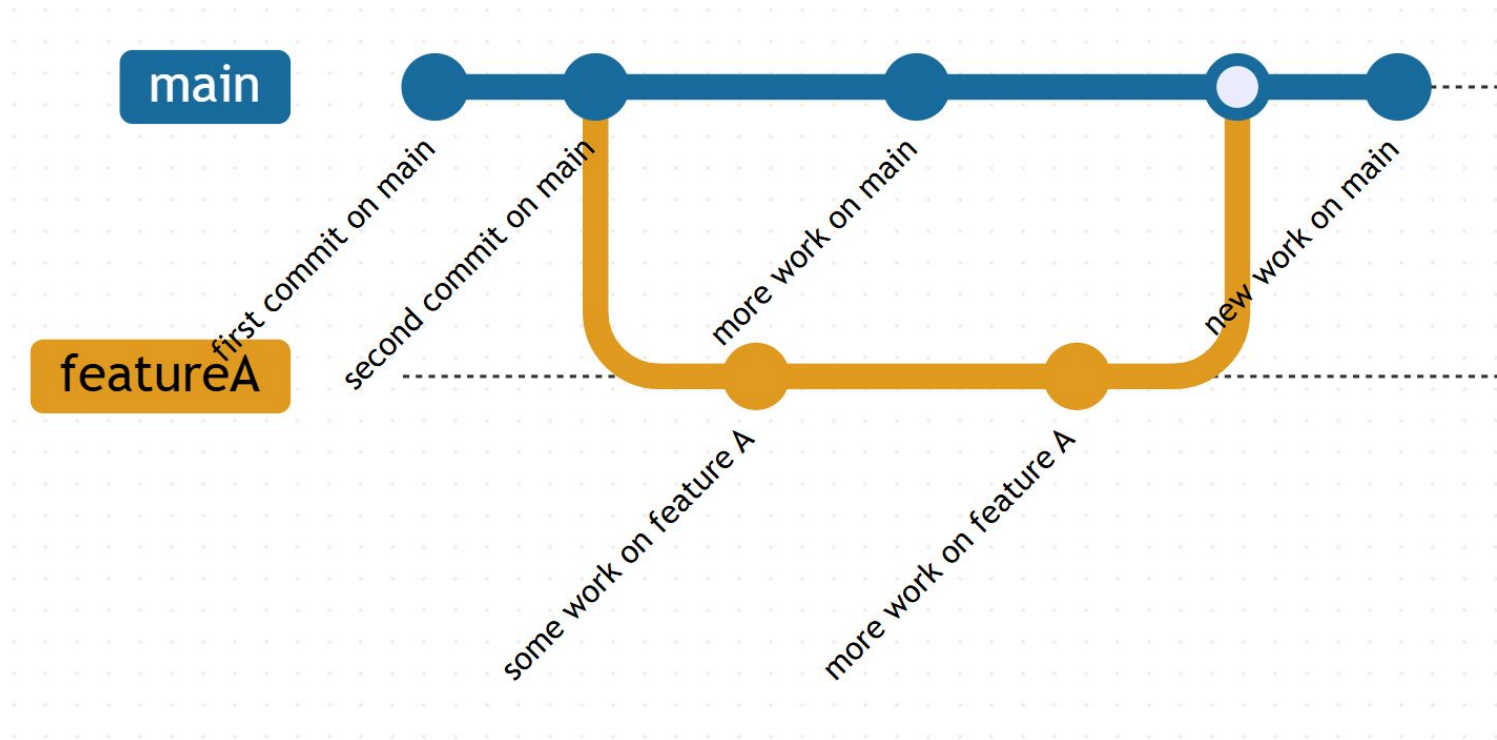
The HEAD pointer is the indicator that allows Git to know which branch you are currently on. When you switch branches, **Git updates the files** in your working directory to match the state of that specific branch, in a similar way to how a stash works.

Branches

A branch should not be viewed as a folder of files (no relation between a folder and branch!), but rather as a **pointer** to a specific commit.

- When you create a branch, it starts exactly where you are currently located. **It shares all previous history with the parent branch** until you introduce new changes.
- The commit lineage: a branch is said to "contain" commits. Technically, it represents a **sequence of snapshots** that **diverge** from the main line. With each new commit, the branch pointer automatically moves forward to mark the new state.
- Isolation and merging: working on a branch **isolates** your modifications. Once your goal is achieved, the **merge** step allows you to **combine** this lineage of commits back into the main/master branch to unify the project.

Graph



Branches: naming

Branches are always named. Here is a common naming convention:

- Default branch, **main** or **master**: Represents the stable state of the project, ready for production.
- Feature branch: Used for developing new features. Example: **feature/auth-login**.
- Bug fix branch: Used for fixing bugs. Example: **bugfix/login-error**.

💡 Note: There are different types of workflows when working with branches, such as **Git Flow**, **Trunk-based development**, and others, which define how and when these branches are created and merged.

Branches: commands

- **git branch <branch-name>**: Creates a new branch from your current location.
- **git switch <branch-name>**: Allows you to switch to the specified branch. You can also use `git checkout <branch-name>`.
- **git switch -c <branch-name>**: Creates the branch AND switches to it immediately (equivalent to **git checkout -b**).
- **git branch**: Lists all local branches and highlights the one you are currently working on.
- **git branch -d <branch-name>**: Deletes a branch (once its work has been merged).

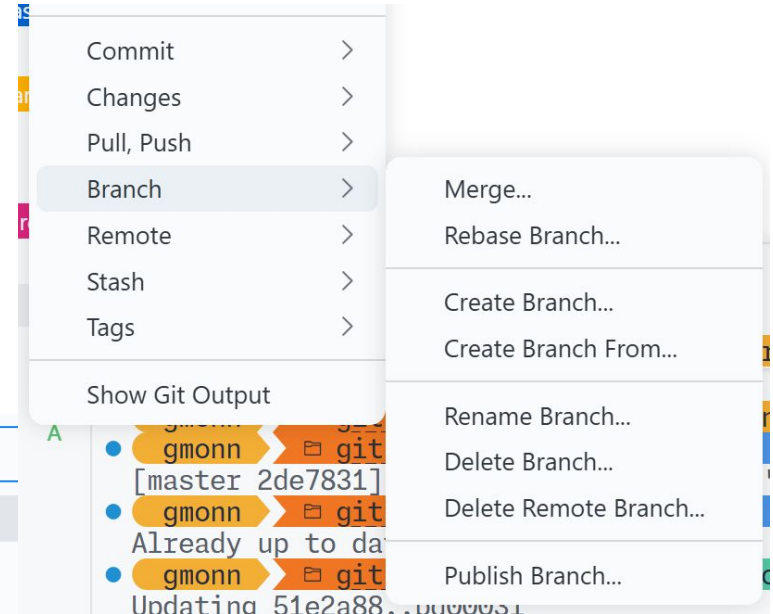
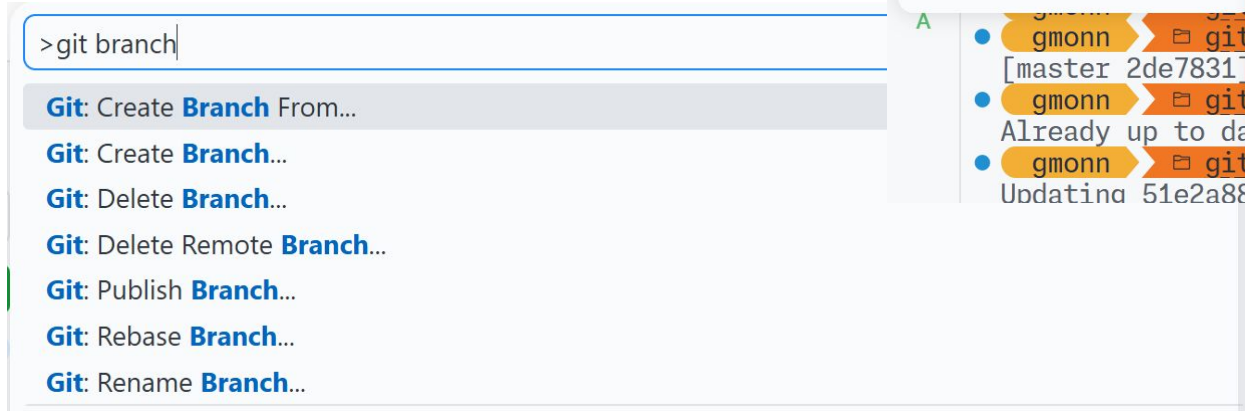


git branch

• gmonn > git-course > master ≠ > ✓ git switch feature1
Switched to branch 'feature1'

• gmonn > git-course > master ≠ > ✓ git branch
featureA
* master

git branch



Exercise 6: isolation and navigating between branches

Objectives: Understand how Git updates your working directory to reflect the state of the active branch.

1. Verify that the repository is clean (no uncommitted changes).
2. Create a new branch named **feature-a** and switch to it using the **git branch** and **git switch** commands.
3. Create a new file named **feature.txt** with the text content of your choice. Commit the file using the **git add** and **git commit** commands.
4. Switch back to the **master** branch: **git switch master**. The file **feature.txt** has disappeared. This is normal: it does not exist in the history of the **master** branch.
5. Return to the **feature-a** branch, and the file should reappear.

Merge

Integrating changes: merge

The **git merge** command allows you to bring two lines of development (branches) together into one.

To merge a working branch (e.g., feature1) into the main branch (main/master):

You must first position yourself on the destination branch: **git switch master**.

Then, launch the merge: **git merge feature1**.



Warning: It is possible to merge any branch into any branch. You must think carefully about the direction of the merge.

Different types of merge

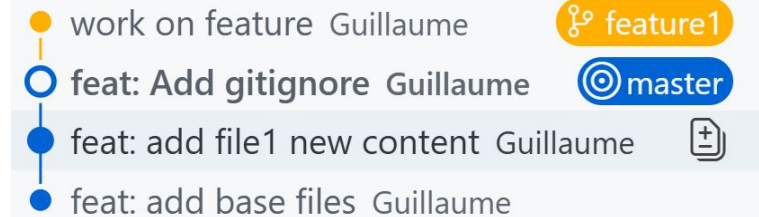
There are different types of merges:

- **Fast-forward:** Possible when the **destination** branch **has not received any changes** in the meantime. The pointer of main/master is simply moved to the last commit of the branch to be merged.
- **3-way merge:** When **changes have also occurred** on the destination branch. A new commit is created.

Merge: fast-forward

This is the simplest case. It occurs when the **master** branch **has not received any new commits** since you created your working branch.

- Git sees that there is a **direct, linear path** between the last commit of master and the one on your branch.
- Instead of creating a new commit, Git simply **moves the master pointer** forward so that it points to the same commit as your branch.
- Result: The history remains a **perfect straight line**. You cannot visually tell that a separate branch ever existed.



Merge: 3-way

This case occurs when **both branches have diverged**: you have made commits on your branch, but in the meantime, other commits have also been added to master.

- Git can no longer simply move the master pointer forward because the **history is no longer linear**.
- Instead, it performs what is called a "3-way merge" by using three snapshots: the common ancestor commit and the latest commits of both branches (destination and source). It then automatically creates a **merge commit**.
- This commit is special because it has **two parent commits**. It seals the reunion of the two independent lines of development.



git merge

```
● gmonn > git-course > feature1 # ✓ git merge feature1
Updating fa9a79e..cac6002
Fast-forward
 feature.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
```


Exercise 7: 3-way merge

Objective: Trigger a divergence to see Git create a merge commit.

1. We already have a **feature-a** branch from the previous exercise. Verify that the file **feature.txt** has been committed to this branch, then switch back to master.
2. Make a modification on **master**, either a new file or a change to one of the existing files. Commit it. Now, your two branches have diverged.
3. Merge the branch into master: **git merge feature-a**
4. Git will open your text editor to ask you to confirm the merge commit message. Save and close it.
5. Visualize the result: **git log --oneline --graph**. You will see the two lines merge back together.

Exercise 7: 3-way merge

```
● gmonn > git-course > master # ✓ git log --oneline --graph
* 904685e (HEAD -> master) Merge branch 'feature-a'
| \
| * cac6002 (feature-a) work on feature
* | 7df7af4 work on master
| /
* fa9a79e feat: Add gitignore
* 2c31761 feat: add file1 new content
* 7113ac3 feat: add base files
```

Merge conflicts

Merge conflicts

A conflict occurs when Git is unable to fusion automatically two changements. This arrives generally when the same line of a same file has been modified in two different ways in the two branches that you are trying to reunite.

How Git signals a conflict?

During a git merge, Git stops and displays a message:

```
⊗ gmonn > git-course > master ≠ > ✓ git merge feature-b
Auto-merging file1.txt
CONFLICT (content): Merge conflict in file1.txt
Automatic merge failed: fix conflicts and then commit the result.
```

⚠ The merge is still in progress! It is possible to cancel it with **git merge --abort**

Merge conflicts

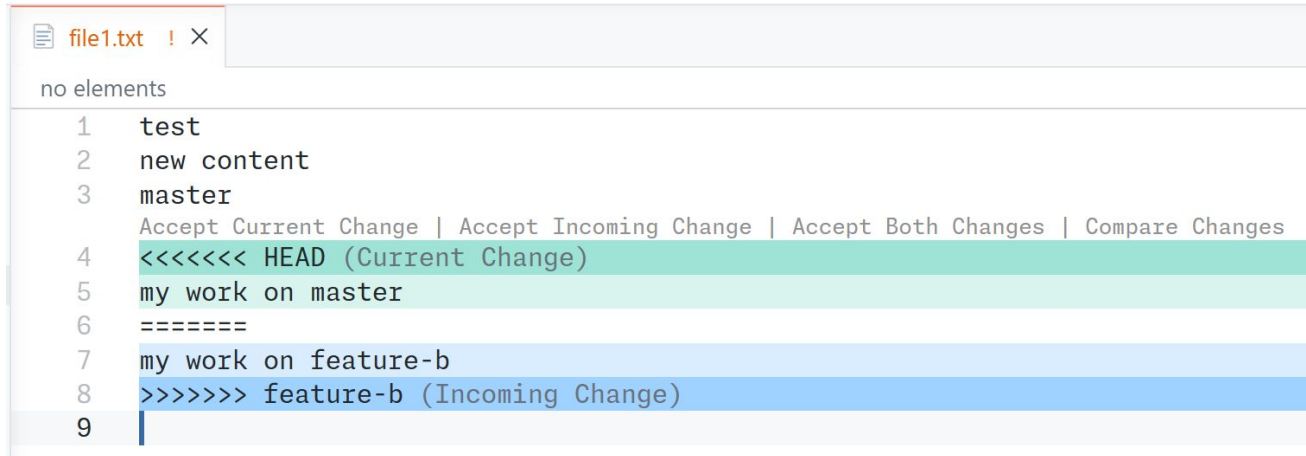
Git inserts markers directly into your files to isolate the two versions:

```
test
new content
master
<<<<<<< HEAD
my work on master
=====
my work on feature-b
>>>>>>> feature-b
```

Resolving conflicts

Today, most IDEs or Git tools (GitKraken, GitHub Desktop, etc.) offer visual interfaces that make decision-making easier.

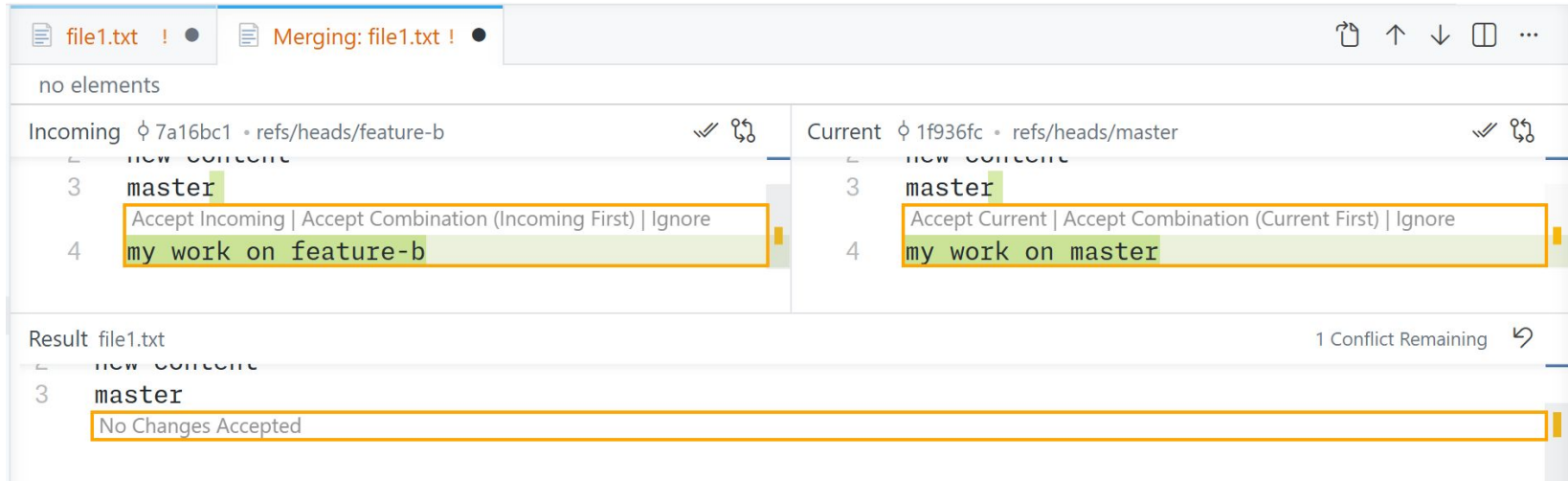
Example: In VSCode, the conflict is color-coded, and clickable links are provided directly above the markers.



```
file1.txt ! X
no elements
1 test
2 new content
3 master
  Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
4 <<<<<< HEAD (Current Change)
5 my work on master
6 =====
7 my work on feature-b
8 >>>>>> feature-b (Incoming Change)
9
```

VSCode: merge conflict editor

VSCode also offers a merge conflict editor:



⚠ Opening the merge editor "takes control" over the conflict resolution, and the conflict markers temporarily disappear from the file (choose one or the other).

Resolving conflicts

Once you have chosen the final version of the code via your tool:

- Save the file.
- Mark as resolved: Add the file to the index with **git add [file-name]**.
- Finalize: Create the merge commit with **git commit**. Git will pre-fill the message to indicate that it is a conflict resolution.

Strategies to reduce conflicts

There is no magic recipe for resolving a conflict. Each conflicting piece of code must be **analyzed** and, in case of doubt, **discussed** with the collaborator who made the other modification.

What can help reduce conflicts:

- **Avoid working on the same code** (and the same branch) simultaneously.
- If a major change is underway, **consider delaying** the other feature that might touch the same code.
- Keep the content of features **as small as possible** and **focused** on a single topic.
- Use **short-lived branches** (branches with a short lifespan, merged as soon as possible).

Exercise 8: resolve a conflict

Objective: Intentionally create a conflict to learn how to manage it.

1. Create a branch: **git switch -c feature-branch**. Modify line 1 of the file **file1.txt**: write "Version A".
2. Commit: **git add file1.txt** then **git commit -m "set version A"**.
3. Switch back to **master**, and modify the same line of the file **file1.txt**: write "Version B". Commit: **git add file1.txt** then **git commit -m "set version B"**.
4. Attempt to merge using the command **git merge feature-branch**. Git announces a conflict.
5. Open the file in VS Code. Choose one of the two versions using the buttons. You can also edit the code yourself, remove the conflict markers, and choose to build a mix of both changes.
6. Save the file, then do a **git add file1.txt**, followed by a **git commit** to complete the merge. The merge commit now contains the chosen version of the text.

Rebase

Rebase: rewriting history

Rebase is a very popular alternative to merging. Instead of creating a "merge commit" to link two branches, **rebase "moves" your entire working branch** to the tip of the main/master branch (or another chosen branch).

Git actually **"replays"** or **"re-commits"** each of your chosen commits one by one onto the destination branch.

After doing this, you simply need to perform a fast-forward merge. The history becomes completely linear.

→ Unlike a merge, which processes everything all at once, a rebase applies your commits **one by one**. This means you might encounter conflicts at multiple stages, and it may be preferable to perform a **squash** (see next section).

→ New commits: The "replayed" commits are actually brand new commits with different SHA hashes.

Rebase: commands

- **git rebase <destination-branch>**: Moves the current branch to the tip of the target branch (usually master).
- **git rebase -i <SHA> or HEAD~n**: Launches an interactive rebase to clean up, merge (squash, see next section), or reorder commits.
- **git rebase --continue**: Once a conflict is resolved and the file is staged (git add), this command tells Git to move on to the next commit in the stack.
- **git rebase --skip**: Ignoring the current problematic commit and moves to the next one (warning: you will lose the changes made in this commit).
- **git rebase --abort**: Completely cancels the rebase operation (just like with a merge). Your branch will instantly return to the exact state it was in before running the command.

Rebase

gmonn > git-course > feature-new > ✓ git rebase master

Auto-merging file1.txt

CONFLICT (content): Merge conflict in file1.txt

error: could not apply 00d1bfe... test

hint: Resolve all conflicts manually, mark them as resolved with

hint: "git add/rm <conflicted_files>", then run "git rebase --continue".

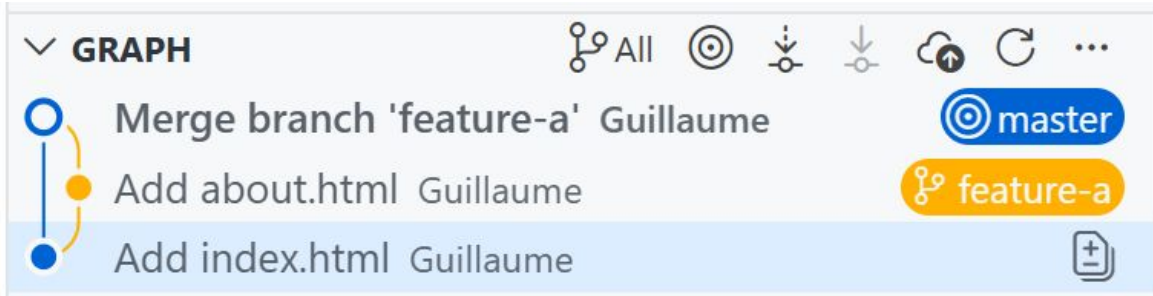
hint: You can instead skip this commit: run "git rebase --skip".

hint: To abort and get back to the state before "git rebase", run "git rebase --abort".

hint: Disable this message with "git config set advice.mergeConflict false"

Could not apply 00d1bfe... # test

Rebase result: merge vs rebase



Rebase: pros and cons

Pros:

- It's really **clean**! The history remains linear. No merge commits, and often just a single commit because a squash is also performed.

Cons:

- **You rewrite history**. Commits may appear recent but are actually old, and they might rely on or update old parts of the code that have since evolved.
- **It does not prevent conflicts**. In fact, without squashing, you can encounter conflicts at every single commit of the rebase, which can become tedious.

Rebase: dangers

Never do this:

- Only **rebase your own branches**, ideally ones that haven't been published yet. If they are published, you must be absolutely sure that you are the only person working on them.
- Always verify which branch you are currently on before running a rebase.
Never rebase master!

Exercise 9: rebase

Objective: Keep a branch up to date with master without creating a merge commit.

1. Ensure you are on **master**. Create a branch called **feature-rebase** and add a commit to it (a new file, for example).
2. Return to master: **git switch master**. Create a new commit on master to simulate project evolution.
3. Return to your working branch: **git switch feature-rebase**
4. Launch the rebase: **git rebase master**
5. Git will move your commits on top of the ones you just made on master. Verify this with **git log --oneline --graph --all**
6. Observe that your **feature-rebase** branch is now located right after the last commit of master, in a straight line.
7. Merge into master: **git switch master** then **git merge feature-rebase**. Since the branches are aligned, Git will perform a simple fast-forward.

Squash

Squash: clean the history

Squashing consists of merging several small commits into a single unique commit in order to keep the history clean.

Example: the 10 small commits like "fix typo", "save", "test 1", etc. can be merged into a single quality commit. For example: "feat: implement shopping cart system".

You generally use interactive rebase (see next section) for this: **git rebase -i HEAD~3** (to process the last 3 commits). An editor opens, and you replace the word pick with squash (or s) for the commits to merge. **You must keep at least 1 pick!**

Squash command

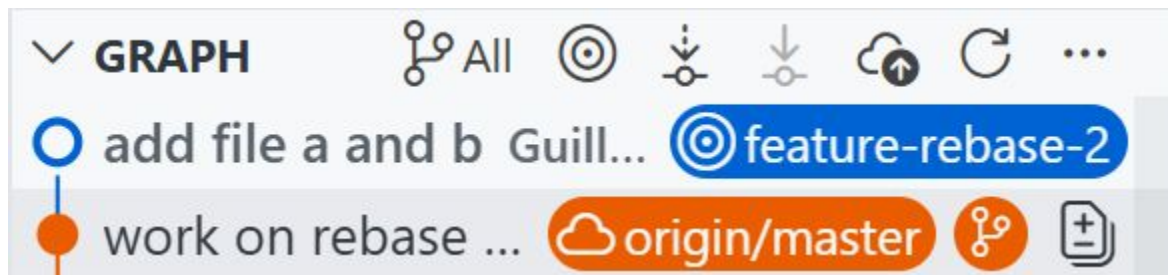
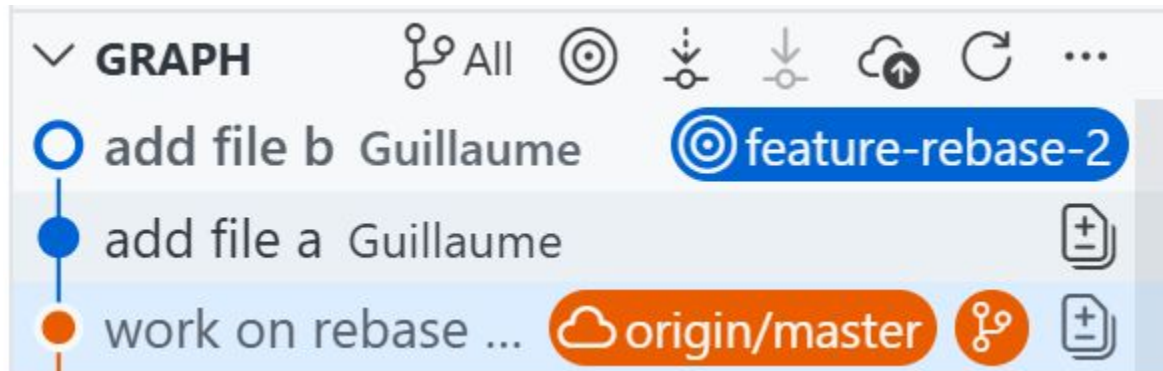
○ gmonn test master # git rebase -i HEAD~2
hint: Waiting for your editor to close the file... □

● gmonn test master # git rebase -i HEAD~2
[detached HEAD db30267] Second commit
Date: Sat May 16 17:29:36 2026 +0200
1 file changed, 1 insertion(+)
Successfully rebased and updated refs/heads/master.

Squash message

```
git-rebase-todo X
no elements
1  pick 66fb9b3 # Second commit
2  pick 4f359e4 # Third commit
3
4  # Rebase a766901..4f359e4 onto a766901 (2 commands)
5  #
6  # Commands:
7  # p, pick <commit> = use commit
8  # r, reword <commit> = use commit, but edit the commit message
9  # e, edit <commit> = use commit, but stop for amending
10 # s, squash <commit> = use commit, but meld into previous commit
11 # f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
12 # ..... commit's log message, unless -C is used, in which case
13 # ..... keep only this commit's message; -c is same as -C but
14 # ..... opens the editor
15 # x, exec <command> = run command (the rest of the line) using shell
16 # b, break = stop here (continue rebase later with 'git rebase --continue')
17 # d, drop <commit> = remove commit
18 # l, label <label> = label current HEAD with a name
```

Squash : before → after



Exercise 10: squash

Objective: Learn how to transform a messy history into a clean commit.

1. Create a new branch: **feature-squash**. Make three successive modifications, creating a commit for each small step (e.g., "commit 1", "commit 2", "commit 3").
2. Verify the history: **git log --oneline**.
3. Launch the interactive rebase: **git rebase -i HEAD~3**.
4. Modify the text file that opens: keep **pick** for the first commit and replace **pick** with **squash** for the next two. Save and close the editor.
5. Write the final message for the single unique commit in the new editor that opens (e.g., "feat: add new file").
6. Verify again: **git log --oneline**. The three commits have disappeared in favor of a single one.

Head and cherry pick

View a specific commit (detached HEAD)

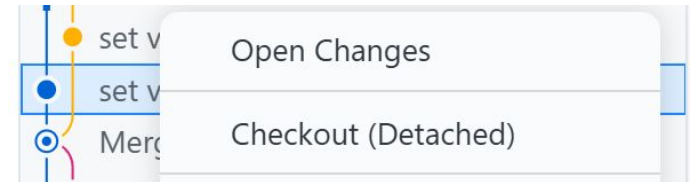
You can ask Git to restore your directory to the exact state it was in during a specific commit.

Command: **git checkout <SHA>** (where <SHA> represents the first characters of the commit).

The "detached HEAD" state: When you check out a specific commit rather than a branch, you enter what Git calls a "detached HEAD" state.

In this state, you can look around and make modifications, but any new commits you create will not belong to any branch. They risk being lost as soon as you switch away.

To return to a branch safely: **git switch master**

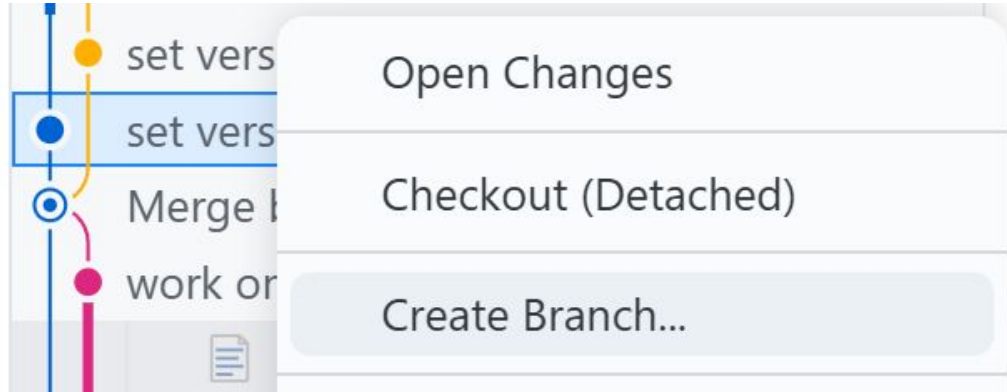


Create a branch from an older commit

This is the ideal solution if you need to fix a bug on an older version.

You start from an old commit to launch a brand new line of development.

Command: **git switch -c <new-branch-name> <SHA>**



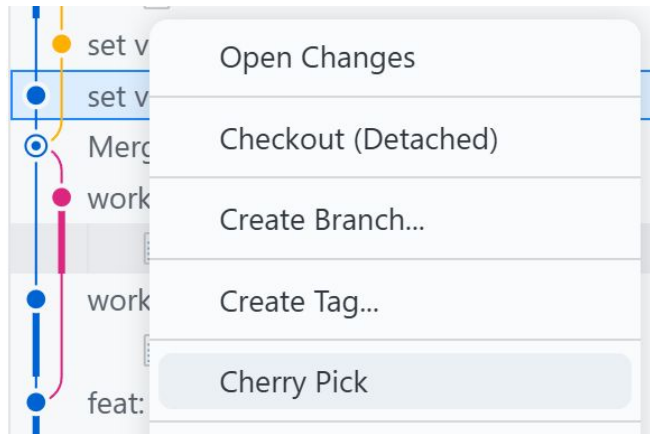
Cherry-pick

cherry-pick allows you to choose a single specific commit from another branch and apply it (like a "copy-paste") onto your current branch.

This is useful when you want to retrieve a bug fix made on another branch without merging the entire branch and its unfinished features.

Command: **git cherry-pick <SHA>**

Note: This creates a brand new commit on your current branch with a new SHA, but with the same content and commit message.



Exercise 11: exploration and cherry-pick

Objectives: Learn how to navigate history and retrieve a specific change.

1. Display history: **git log --oneline**. Choose a commit from earlier in the course and note its SHA.
2. Place HEAD on this commit: **git checkout <SHA>**. Your files should return to the state they were in at that time.
3. Return to the main branch: **git switch master**
4. Create a branch named **old-version-test** from an old commit: **git switch -c old-version-test <SHA>**. Verify that the branch indeed contains the old code.
5. Return to master: **git switch master** and create a new branch called **temp-fix**. Create a file named **fix.txt**, commit it, and note the SHA of this commit.
6. Return to **old-version-test**: **git switch old-version-test**
7. Apply only this fix to **old-version-test**: **git cherry-pick <SHA>**
8. The file **fix.txt** is now on your **old-version-test** branch. However, because of its recent creation date, this brand-new commit will be placed right at the top of your branch's history.

Reset and amend

git commit --amend

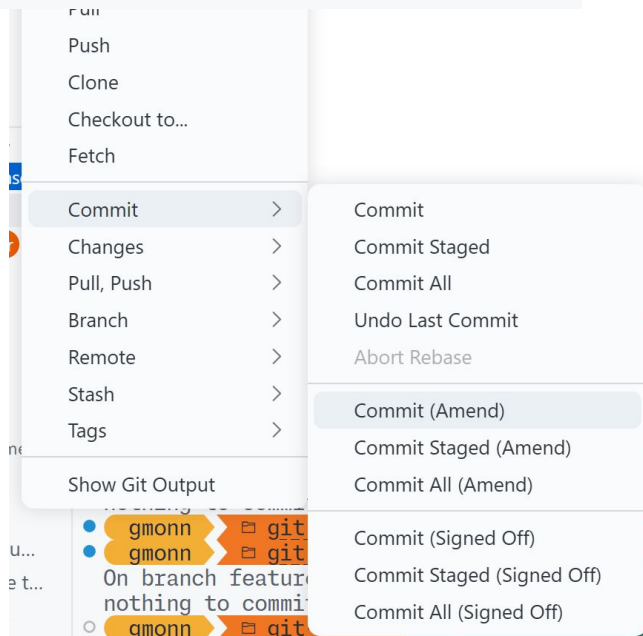
The **amend** tool is perfect for correcting the **last commit** made on a branch.
Some use cases:

- Forgetting a file.
- Typo or incomplete commit message.
- Error in the code.

Command: **git commit --amend -m "New message"**

git commit --amend

```
gmonn git-course feature-rebase-2 # git commit --amend -m "new message"
[feature-rebase-2 a9f7aff] new message
Date: Sun May 3 18:18:56 2026 +0200
1 file changed, 1 insertion(+)
create mode 100644 note.txt
```



git reset: undo some commits

reset allows you to "**undo**" **commits** or empty the staging area (index). There are several modes, but we will focus on the two most used:

- **Soft (--soft)**: Cancels the commit but keeps your modifications in the index (ready to be re-committed). This is ideal for rewriting a commit message or grouping changes together.
- **Mixed (--mixed)**: Default mode. Cancels the commit and removes the files from the index (appearing in red in git status). Your modifications are still in your directory, but you must do a git add again to re-stage them.
- **Hard (--hard)**: Cancels everything. The commit disappears and your modifications are permanently deleted. Your directory returns to the exact state of the targeted commit.

Examples of commands:

- **git reset --soft HEAD~1**: cancels the last commit.
- **git reset --hard <SHA>**: returns to a specific past state.

 [Documentation](#)

git reset

```
gmonn > git-course > feature-rebase-2 # > ✓ git reset --hard HEAD~1  
HEAD is now at a9f7aff new message
```

Exercise 12: reset and amend

Objective: Master fixing commits and the different levels of undoing.

Amend:

1. Create a file **note.txt** with the text "Version A". Add and commit: **git add note.txt** and **git commit -m "feat: initial note"**.
2. Modify the file to write "Version B". Add to staging: **git add note.txt**. Merge with the previous commit: **git commit --amend --no-edit**. Verify with **git log**: there is still only one commit.

Reset:

3. Create a file **temp.txt**, add it, and commit it. Cancel this commit while keeping the files in the index: **git reset --soft HEAD~1**
4. **git status** shows the file in green (tracked).
5. Re-make the commit, then use the default reset (mixed): **git reset HEAD~1**
6. **git status** shows the file in red (untracked).
7. Re-add the file to staging and commit. Cancel everything to return to the previous commit: **git reset --hard HEAD~1**

Tags

Tags

While branches are mobile and evolve with each commit, a **Tag** is a **fixed marker** in history. Think of it as a "label" permanently pinned to a specific commit.

Why use Tags?

- **Versioning (Releases)**: This is the main use case. We mark stable versions of the project (e.g., v1.0, v2.1.4).
- **Milestones**: Mark an important step in the project (e.g., beta-launch, final-submission).
- **Stability**: A tag never moves. If you add commits after adding a tag, the tag remains on the original commit.

The two type of tags

Lightweight tag:

- It is just a name pointing to a commit.
- Command: **git tag <name>**

Annotated tag:

- It is a complete object in the Git database containing the creator's name, the date, and a tag message (similar to a commit message).
- Command: **git tag -a <name> -m "message"**

Commands

- **git tag**: Lists all the tags in the project.
- **git show <tag-name>**: Displays detailed information about the tag and its associated commit.
- **git tag -d <tag-name>**: Deletes a tag locally.

Tags

▼ **GRAPH** All ...

○ fix Guillaume old-version-test

● fix Guillaume temp-fix

○ Merge branch 'feature-bra...' master

● set version a Guillaume... feature-branch

● set version B Guillaume

Guillaume, 5 minutes ago (April 30, 2026 at 8:26 PM)

fix

1 file changed, 1 insertion(+)

temp-fix v1.0.0

6317ae5 |

● gmonn git-course master # ✓ git tag

v1.0.0

v2.0.0

Exercise 13: marking your versions

Objective: Learn how to place fixed tags in your history.

1. Go to **master**. Create an annotated tag to mark your last exercise using the command: **git tag -a v1.0.0 -m "End of basic training"**
2. Display history: **git log --oneline**. Locate the SHA of the very first commit. Add a lightweight tag to this specific commit with the command: **git tag initial-setup <SHA>**
3. List the tags to verify they are there: **git tag**
4. Display details for version 1.0.0: **git show v1.0.0**
5. Just like with commits, it is possible to move to a tag: **git checkout v1.0.0**
6. Return to the present: **git switch master**

Remote repositories

Remote repositories

A remote repository is a version of your project hosted on the Internet or a network. This is what allows you to **back up your work** off your machine and **collaborate** with other developers.

It is possible to [host Git on a server](#) or use a turnkey solution (self-hosted or not) which can also offer other options (ticketing systems, etc.): [GitHub](#), [GitLab](#), [Gitea](#), [Codeberg](#).

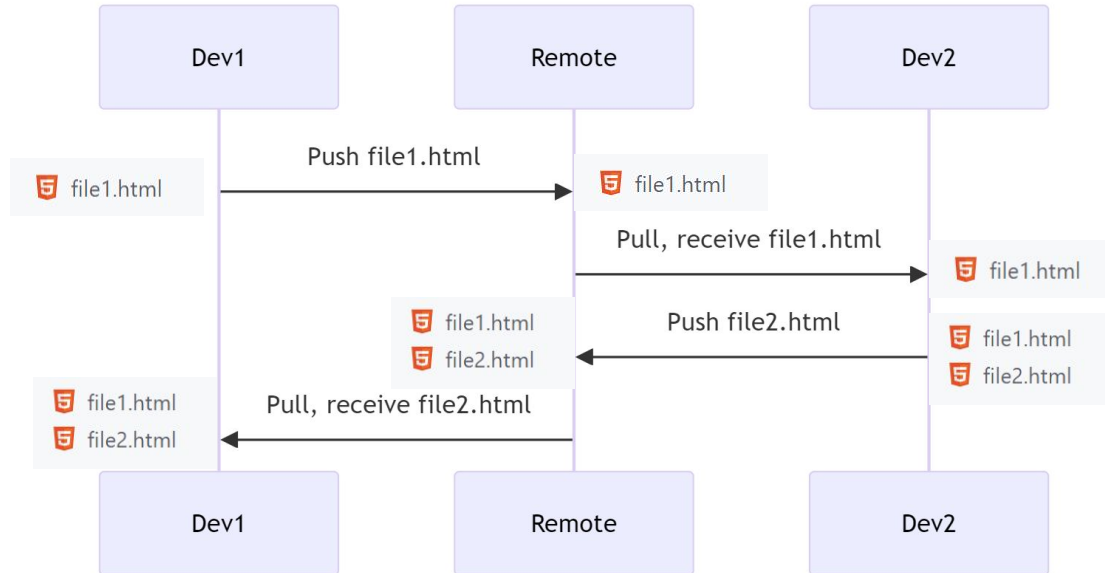
Remote

We generally "connect" one (or more!) remote servers to our local repository. The remote is often considered the "source of truth".

Key Concepts:

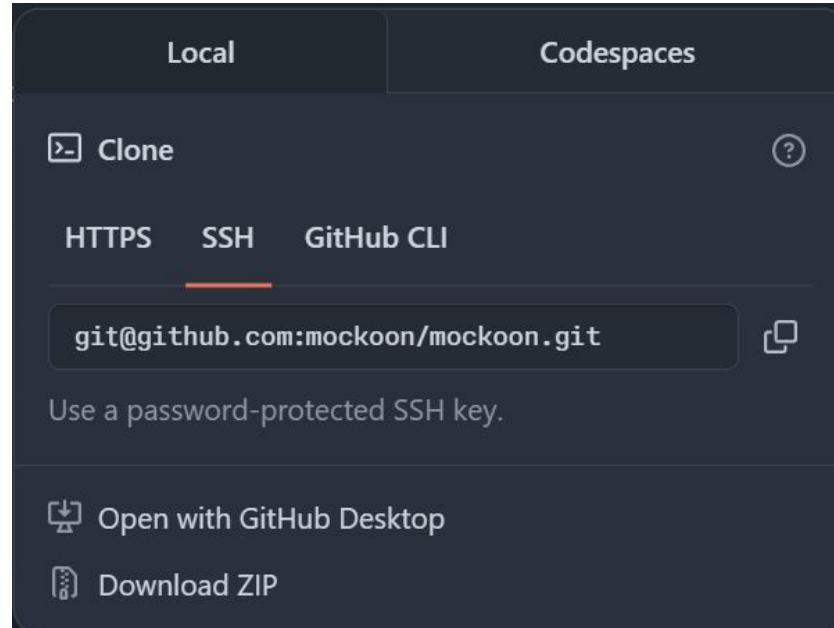
- **Origin:** This is the default **name** given to the **primary remote server**.
- **Clone:** **Creating a complete local copy** of an existing remote repository.
- **Push:** **Sending your local commits** to the remote server.
- **Fetch:** **Retrieving the latest updates** from the server without merging them into your code.
- **Pull:** **Retrieving the latest updates** AND immediately **merging** them into your local code.

Sync with remote



Communicating with remote

Communication between the local Git repository and the remote server can generally be done in two ways: either via **HTTPS** or via **SSH**.



Remote: commands

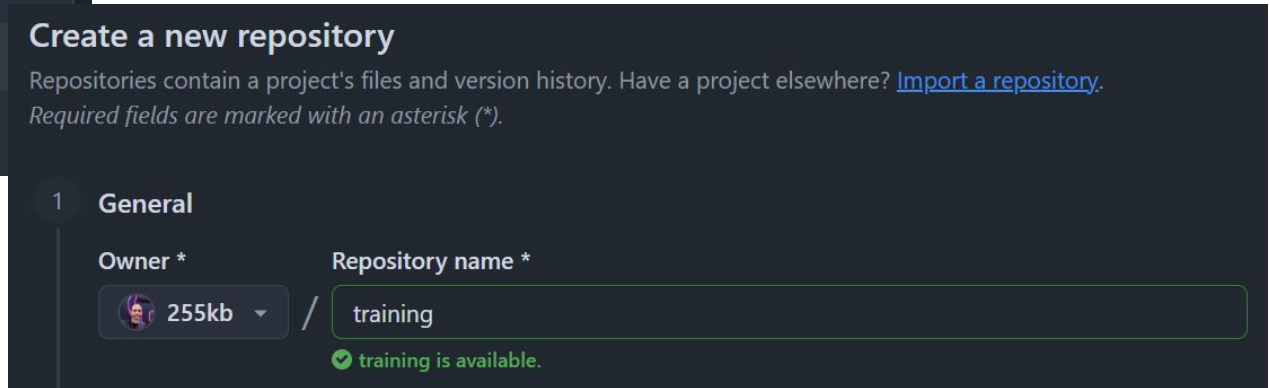
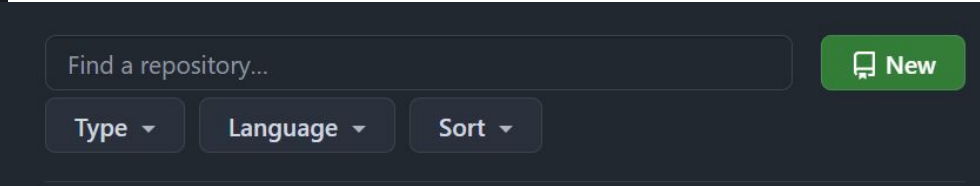
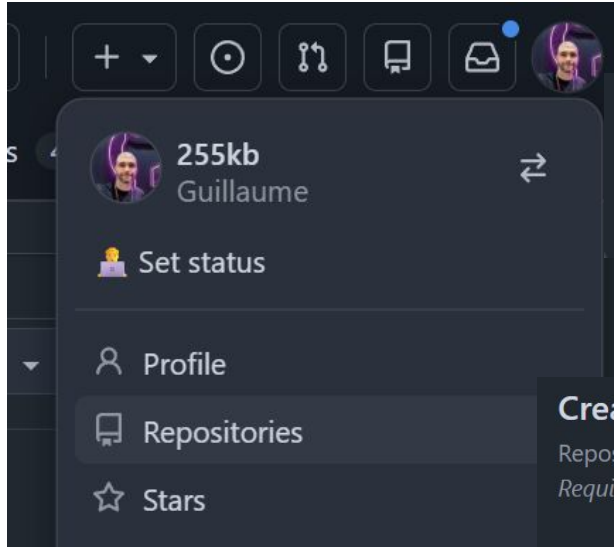
- **git remote add origin <URL>**: Link your local directory to an empty repository created on GitHub. "Origin" is the name given to the remote repository locally. You can name it differently.
- **git push -u origin master**: Send your files for the first time and link your local branch to the remote branch.
- **git clone <URL>**: Download an existing project to start working on it.
- **git pull**: Update your directory with your colleagues' work.
- **git push**: Send your work to the remote repository.

Add or clone

Two approaches are possible when interacting with a remote repository:

- Either the project already exists and we need to work on it, in which case it must be cloned: **git clone <URL>** (URL: HTTPS or SSH). Git automatically creates a folder containing the project files and the local .git repository.
- Or we are the initiator of the project and we have started working locally, in which case we can add a remote repository: **git remote add origin <URL>**

GitHub repositories



Exercise 14: add an SSH key

Objective: Add an SSH key to your GitHub account (if not already done) in order to communicate with your remote repository.

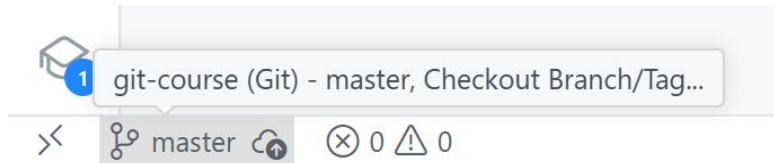
1. Verify if an SSH key already exist on your machine: look in your personal directory with the command: **ls -al ~/.ssh**
If you see a file named **id_ed25519.pub** or **id_rsa.pub**, you already have a key.
2. [Optional] Generate a new SSH key in your terminal using the following command:
ssh-keygen -t ed25519 -C "email@example.com"
3. Copy the content of your new SSH key's **id_ed25519.pub** file, or of the existing key: **cat ~/.ssh/id_ed25519.pub**, to display its content, they copy.
4. Add it to your GitHub account: Go to your account settings (Account settings → SSH and GPG keys → New SSH key). Give a name to the key (for example: "laptop" or "training-vm"). Paste the content of the SSH key and validate.

 [GitHub docs](#)

Exercise 15: setup a remote repo

Objective: Back up your work to a remote server.

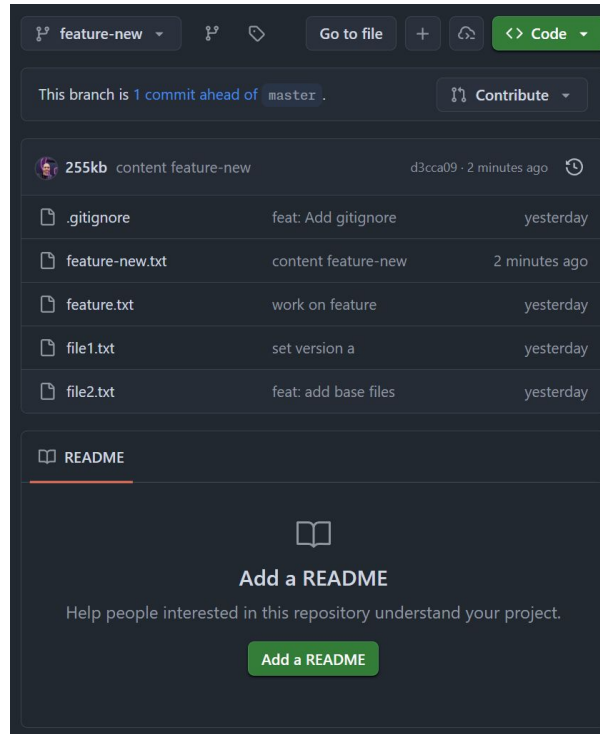
1. Create an empty repository (public or private) on your GitHub account (the name does not matter). Copy the HTTPS or SSH URL of the repository.
2. Add the remote server to your local project with the command **git remote add origin <SSH_URL>** or via the VSCode menu (Remote -> Add remote).
3. Verify the connection with **git remote -v**
4. Send your work with **git push -u origin master** or via the "Publish" button (cloud with the arrow) in VSCode.



5. Refresh the page in your browser to see your files appear online. You can see that the master branch exists on GitHub, but the other branches do not.

GitHub: files and homepage

GitHub allows you to navigate through your code, as well as edit or create files:



Remote branches lifecycle

It is crucial to understand that a branch on your computer and its version on GitHub are **two distinct entities**.

- **Independence**: When you create a branch locally, GitHub (or any other server) knows nothing about it. You must explicitly "**publish**" it.
- **Divergence**: A branch `feature-A` on your machine can be 3 commits ahead of the `feature-A` branch on the server, while your colleague's `feature-A` branch is 2 commits behind it.
- **Tracking**: Git establishes a **relationship** between your **local** branch and the **remote** branch (called the upstream). This allows Git to state: "branch X is 2 commits behind".

→ There is not really any difference between comparing a local branch and a remote one, two local branches, two remote branches, or simply two commits, since a branch is nothing more than a pointer!

Compare local and remote

Before merging changes or sending your work, it is crucial to understand the synchronization status of your branches. Git does not automatically update your view of the server; you have to query it:

- **git fetch**: This command downloads information from the server (Remote) without modifying your working files. It lets you know if changes have occurred and been published.
- **git status**: Once the fetch is complete, git status tells you precisely where you stand:
 - "Your branch is up to date with 'origin/master'": Everything is synchronized.
 - "Your branch is ahead of 'origin/master' by 2 commits": Commits are waiting to be published (push).
 - "Your branch is behind 'origin/master' by 3 commits": The server is ahead, it is time to pull.
- **git diff <branch-name> origin/<branch-name>**: To see exactly which lines of code differ between your branch and the server's branch. This displays the changes line by line.

Compare local and remote

```
gmonn > git-course > test ≡ ✓ git fetch
gmonn > git-course > test ↓1 ✓ git status
```

On branch test

Your branch is behind 'origin/test' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)

nothing to commit, working tree clean

```
gmonn > git-course > test ↓1 ✓ git diff test origin/test
```

```
diff --git a/test-file.txt b/test-file.txt
index 30d74d2..68cf335 100644
```

```
--- a/test-file.txt
```

```
+++ b/test-file.txt
```

```
@@ -1,2 @@
```

```
-test
```

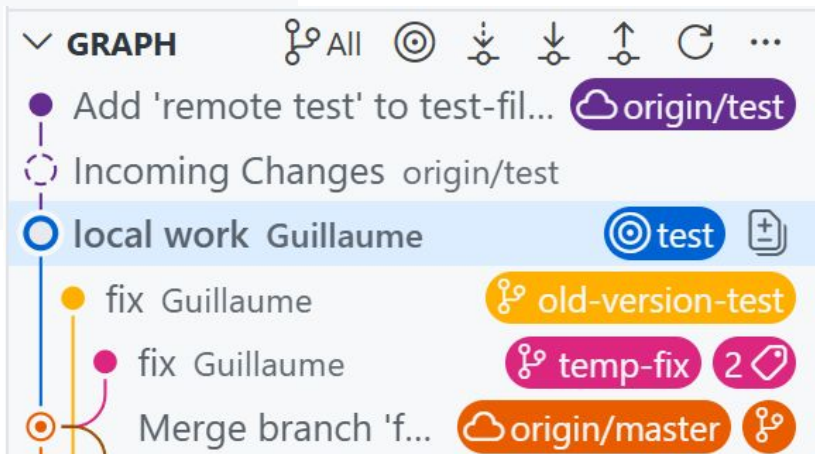
```
\ No newline at end of file
```

```
+test
```

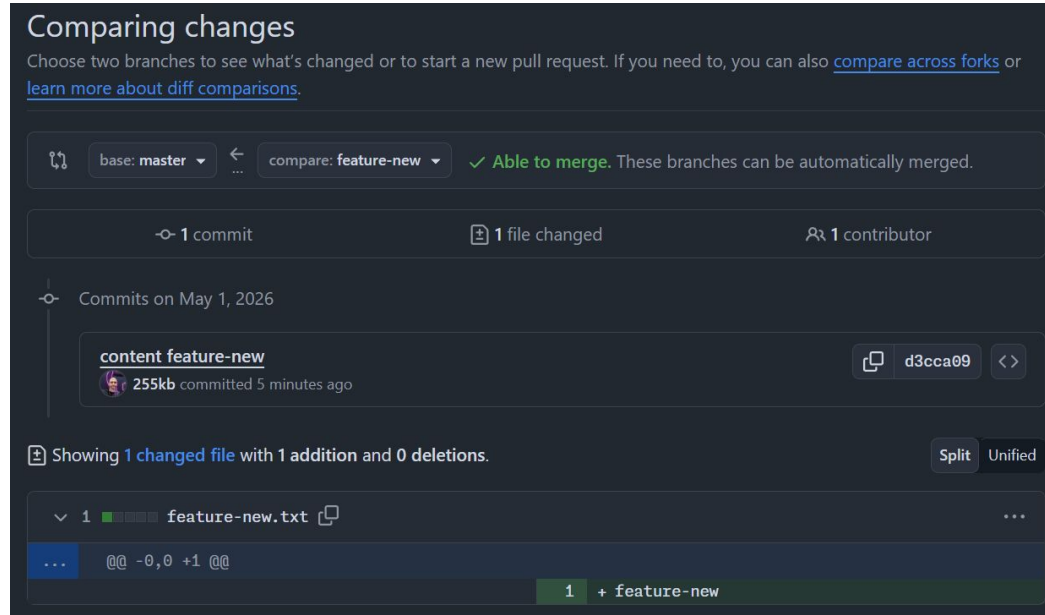
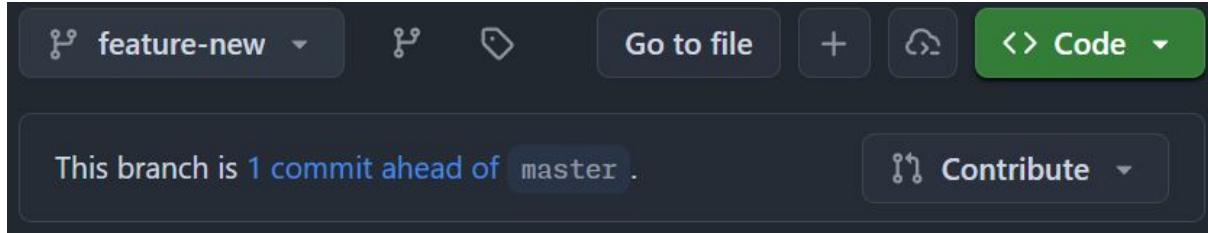
```
+remote test
```

git-course (Git) - Pull 1 commits from origin/test

test ↻ 1↓ 0↑ ⊗ 0 ⚠ 0



Compare remote on GitHub



Sync with remote

Sync with remote

- **git push -u origin <branch-name>**: Publish a new local branch to the server and establish the tracking link.
- **git push**: Once the link is established, this simple command is enough to send new commits.
- **git fetch**: Update your local view of what is happening on the server (without modifying your files).
- **git pull**: Download the changes AND merge them into your local branch (essentially a git fetch followed by a git merge).

Exercise 16: publish and synchronize a branch

Objective: Share a specific branch and update the local master branch.

1. Create a branch **feature-new** and add a commit inside it (either a new file or a new line in an existing file).
2. Publish this branch to GitHub: **git push -u origin feature-new**
3. Verify on GitHub that the branch has correctly appeared in the "Branches" dropdown menu.
4. Return to the master branch locally: **git switch master**
5. Simulate a remote change: On GitHub, create a README.md file (using the green button) while ensuring you are on the **master** branch, and commit the changes.
6. Retrieve this change locally: Run **git fetch** and then **git pull origin master**

Pull, merge and conflicts with remote

A **git pull** is in reality a **git merge**. If you pull a branch that has been modified on remote, but you have made no changes, Git will do a fast-forward merge.

On the other hand, if you have worked on the same branch as someone else, two classic scenarios occur:

- **Pull with merge commit:** you have made a commit locally and your colleague has made a different commit pushed to GitHub (on the same branch), Git will have to create a Merge Commit to link these two histories during the pull.
- **Pull with conflict:** you have modified the same line as your colleague who published to GitHub, the git pull will stop. You will have to resolve the conflict locally before you can send the clean result back to the server.

Exercise 17: managing a pull conflict

Objective: Learn how to reconcile your work with that of the server in the event of a collision.

1. Return to the **feature-new** branch locally and make a modification in the file. Commit but do not push.
2. Go to GitHub, switch branches, and modify the exact same line in the same file with different text. Validate the commit on the web.
3. Run the commands **git fetch**, then **git status**, and **git diff feature-new origin/feature-new**, or visualize the VSCode graph which should show a divergence ("Your branch and 'origin/feature-new' have diverged, and have 1 and 1 different commits each").
4. Attempt to retrieve the work from the server: **git pull**
5. The conflict should open in VSCode. Resolve the conflict by choosing the final version.
6. Complete the merge: **git add** and **git commit**. Finally, send the merged result to the server: **git push**

→ You can see that a merge with a merge commit has been performed.

Best practices

To prevent Git from becoming a headache, there are simple daily habits you can adopt:

- Most tools like VSCode perform periodic **git fetch** operations. Always check your graph before starting to work and run a **git pull** if necessary.
- Run regular **git push** commands to back up your work, share it with the rest of the team, and trigger CI/CD scripts (tests, deployment, etc.).
- **One branch = one topic, one commit = one topic**. Keep your commits clean and write meaningful commit messages.
- **Avoid working and especially committing directly on master** (this is often strictly forbidden). It must always remain stable and ready to be deployed.
- **Use short-lived branches**: A branch should only live for the duration of a specific task (from a few hours to a few days). The longer a branch lives without being merged, the higher the risk of a "monstrous" conflict.
- **Understand Git operations**: Always double-check which branch you are merging or rebasing onto another. Are you moving $A \rightarrow B$ or $B \rightarrow A$? (Be especially careful with rebases on the wrong branch!)

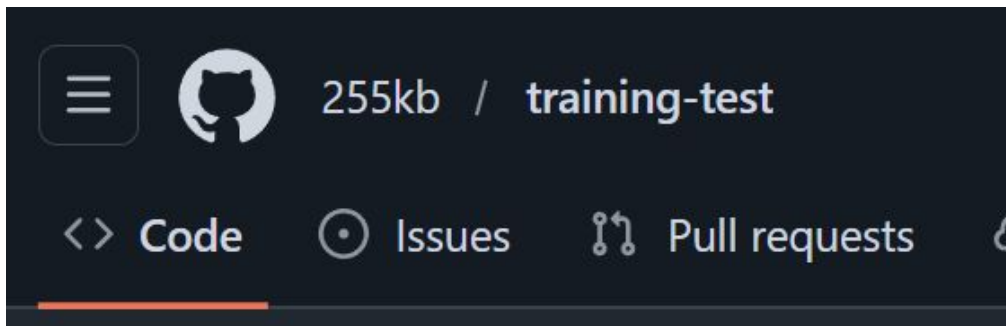
GitHub collaboration

Users vs organizations

Collaboration on GitHub is done either on a personal repository, to which collaborators are invited, or on an organization's repository, of which several Git users can be members.

In both cases, the URLs are similar:

github.com/{user_name|org_name}/{repo_name}



Tools

Several tools are offered to manage the development lifecycle:

- **Issues:** A classic ticketing system featuring labels, milestones, assignees, and more.
- **Pull requests:** A "merge request" system structured similarly to issues.
- **Discussions:** A forum dedicated to conversation, questions, or managing project support.
- **Wiki:** A built-in documentation system using Markdown.
- **Projects:** Kanban boards designed to manage issues, which also include charts (like burndown charts) and a roadmap system.

Issues

[Code](#) [Issues 1](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

[Labels](#) [Milestones](#) [New issue](#)

☐ **Open 1** ☐ **Closed 0**

Author ▾ Labels ▾ Projects ▾ Milestones ▾ Assignees ▾ **↕ Newest ▾**

☐  **Do something** [bug](#) [documentation](#)

#1 · 255kb opened now 

[Give feedback](#)

Discussions

[Code](#) [Issues 1](#) [Pull requests](#) [Agents](#) [Discussions](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

Sort by: Latest activity Label Filter: Open [New discussion](#)

Categories

View all discussions

Announcements

General

Ideas

Polls

Q&A

Discussions

1

How can I do this?

255kb asked now in Q&A · Unanswered

0

Wiki

<> Code Issues Pull requests Agents Actions Projects **Wiki** Security and quality Insights Settings

Home

Edit

New page

Guillaume edited this page now · [1 revision](#)

Welcome to the training-test wiki!

+ Add a custom footer

Pages 1

Find a page or section...

[Home](#)

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/255kb/trainin>



Projects

Mockoon's public roadmap

Roadmap

+ New view

Filter by keyword or by field

Planned (next major release) 2

Issues planned for the next major release

mockoon #1773

Node 18 out of support

component:cli

component:cloud

component:lib-commons-server

component:lib-serverless

effort:easy

priority:low

requires:documentation

requires:major

mockoon #2079

[RFC] Serverless lib deprecation

component:lib-serverless

effort:easy

priority:low

requires:major

status:need-feedback

Planned 4

mockoon #1059

Support multi-select, batch delete, batch toggle, and group disable for URLs

component:app

effort:medium

priority:low

status:approved

mockoon #1395

Add an undo/redo feature

component:app

effort:hard

priority:low

status:approved

mockoon #1996

Improve OpenAPI parser resulting mock file size

Analysis/POC 1

mockoon #1814

add mockoon app to flathub.org

component:dx

effort:medium

pending:help-wanted

priority:low

status:need-feedback

status:under-consideration

In progress 7

mockoon #1909

Unable to launch Mockoon after v9.4.0

component:app

effort:easy

priority:medium

status:app

mockoon #1045

Export logs as HAR

component:app

effort:med

status:approved

mockoon #1792

Feature Request - Add defa

Copyright 1kb SARL-S - contact@1kb.software - 1kb.software

Pull requests

GitHub offers a system called a "pull request". When a branch is published, it is possible to create a pull request, a sort of merge request (similar to issues) that provides an enhanced view of the changes:

- **Changes:** A view of the differences in the code, file by file.
- **Review requests:** It is possible to request and assign multiple people to review the code.
- **Comments:** It is possible to comment on the changes line by line and propose alternative solutions.
- **Conflict resolution:** The online interface allows you to resolve conflicts directly. However, they can also be resolved locally on your machine (as done previously).
- **Merge/Rebase:** GitHub handles the merge on the remote side. This is often the preferred solution rather than performing a local merge.

Pull requests

[Code](#) [Issues 1](#) [Pull requests 1](#) [Agents](#) [Discussions](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

Feature new #3

🔓 Open 255kb wants to merge 4 commits into `master` from `feature-new` 

💬 Conversation 0 🔗 Commits 4 📋 Checks 0 📄 Files changed 1 +1 



255kb commented now Owner ...

No description provided.



 Mention @copilot in a comment to make changes to this pull request.

 **255kb** added 4 commits 12 hours ago

-   `content` `feature-new` d3cca09
-   `feature new` `local change` 162c408
-   Update `feature-new.txt` with remote test Verified 345256c
-   Merge branch 'feature-new' of github.com:255kb/training-test into fea... 8019e34

Reviewers

 **Copilot** ●

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Pull requests

Feature new #3

 Open 255kb wants to merge 4 commits into `master` from `feature-new` 

 Conversation 0

 Commits 4

 Checks 0

 Files changed 1

 All commits 

0 / 1 viewed Submit review     0

 feature-new.txt

 feature-new.txt 

...

@@ -0,0 +1 @@

1

+ test remote

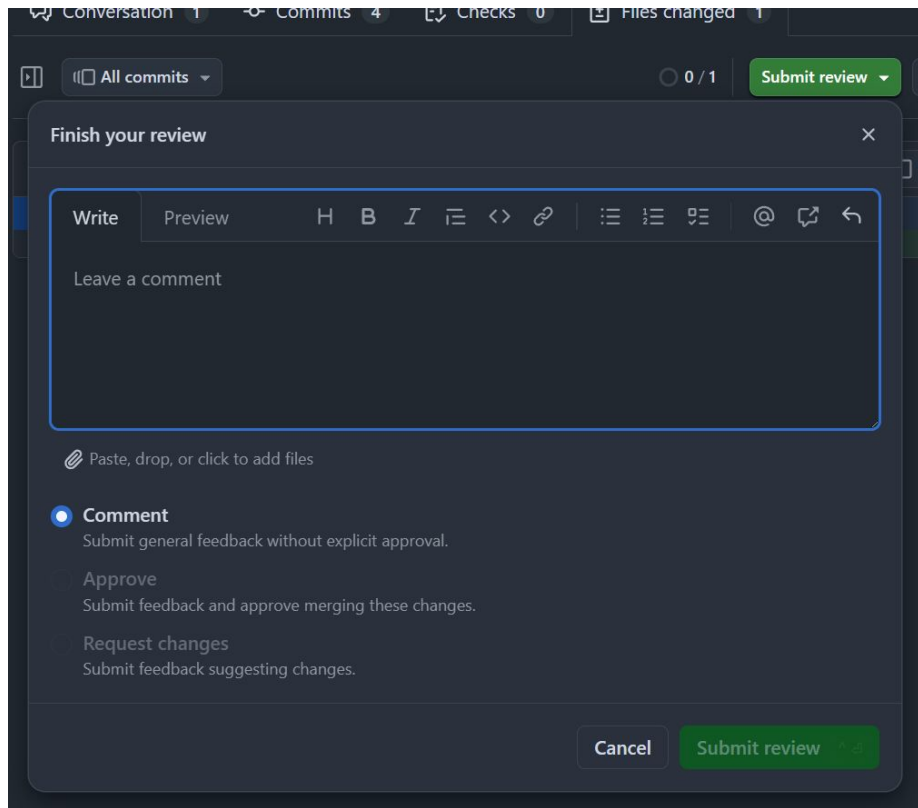
+1 



Viewed 

 ...

Pull requests: reviews



Pull requests: resolving conflicts

 **This branch has conflicts that must be resolved**
Use the [web editor](#) or the command line to resolve conflicts before continuing.

 file1.txt

Resolve conflicts ▾

Merge pull request ▾

You can line. [View](#)

Feature new #3

Resolving conflicts between `feature-new` and `master` and committing changes → `feature-new`

1 conflicting file

 file1.txt
file1.txt

file1.txt

1 conflict ⓘ Prev ^ Next ▾ ⚙ ▾ Mark as resolved

```
1  version A
2  new content
   Accept current change | Accept incoming change |
   Accept both changes
3  <<<<<< feature-new (Current change)
4  masteraaaa
5  =====
6  masteroiouio
7  >>>>>> master (Incoming change)
8
```

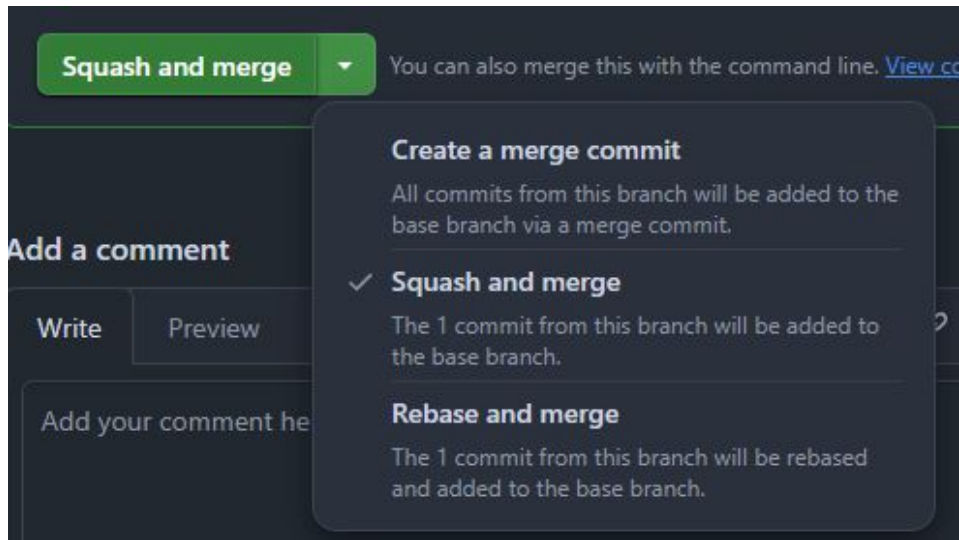
Pull requests: tips

- A pull request is **automatically updated** when you **push** to the branch chosen for the PR.
- **Comments** can be posted directly **on the PR** or **on specific snippets of code**.
- It is possible (and recommended) to **link a PR to an issue** by mentioning the issue number in the description of the PR or within one of the commits (e.g., **#1234**).
- Certain keywords will automatically **close an issue** when the PR is merged: "Close #1234", "Fixes #1234", "Resolved #1234", etc. ([docs](#)).

Pull requests: merge/rebase

GitHub offers 3 merge modes for pull requests:

- **Merge commit:** a "classic" fast-forward or 3-way merge.
- **Squash and merge:** squash the commits and merge (fast-forward or 3-way).
- **Rebase and merge:** rebase on the destination branch and fast-forward merge.



Exercise 18: collaborate on GitHub

Objective: Create a 3-page website with two or three people, use pull requests, and intentionally trigger/resolve a conflict.

1. Setup (⚠ only one developer): **Create a public repository on GitHub** (you can choose the name) and **check the "Add readme" box**. This ensures there is already an initial commit and a **main** branch (the default name on GitHub).
2. Invite the other developers: Go to Settings > Collaborators > Add people.
3. Clone the repository: All developers must **clone** the repository using the SSH URL (Code tab, green "Code" button → Local → SSH).

The Goal: The aim is to create a mini website with a few pages linked by a navigation menu (for example: **index.html**, **about.html**, and **features.html**). You can create more or fewer pages; the content does not matter, except for the navigation menu, which will make it easier to trigger one or more conflicts.

4. Set up issues to list the tasks to be accomplished and assign them among yourselves.

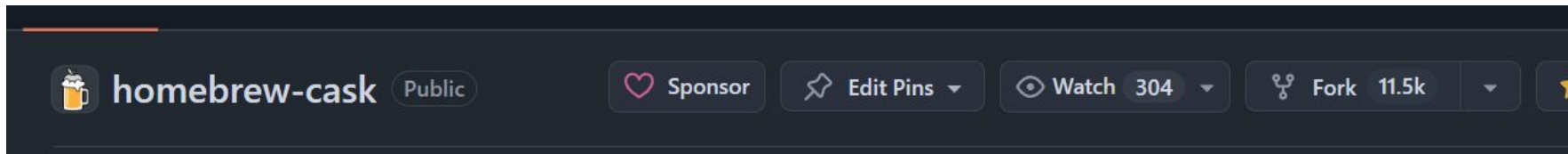
Exercise 18: collaborate on GitHub

5. Start working! Each developer creates a local branch for their issue, commits their changes, and pushes. Then, they open a PR (pull request) and assign someone else as a reviewer.
6. The reviewers can leave comments or approve. After approval, it is generally the responsibility of the developer who opened the PR to merge it using the "Merge pull request" button.
7. The other developers can now pull the new version of main because the merge was executed on the remote, and synchronization is not automatic.
8. If you haven't encountered a conflict yet, try to create one—perhaps by each modifying the navigation to add a link to the new page—and resolve it either online, or by pulling and merging locally, followed by a push.

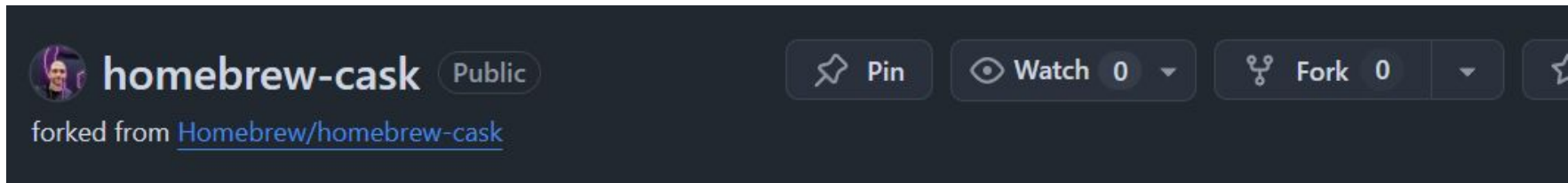
Contribute to open-source on GitHub

Contribute on GitHub

GitHub facilitates contributions to third-party repositories. In order to contribute, the preliminary step is to fork the repository:



This will create a copy of the repository under the username or organization name:



Contribute on GitHub

Then, you simply need to work on the "forked" repo, by creating a branch and publishing it (push). No difference.

The final step consists of opening a pull request **pointing toward the original repository**, instead of your own copy:

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

 base repository: Homebrew/homebrew-cask ▾

base: main ▾

←
...

head repository: 255kb/homebrew-cask ▾

compare: 255kb-patch-1 ▾

✓ **Able to merge.** These branches can be automatically merged.

The GitHub platform

The GitHub platform

GitHub offers many tools beyond git repository management and project management (issues, projects, etc.):

- **Branch protection:** adding rules to prevent, for example, a direct push to main/master.
- **Vulnerability/secret/code quality scanning:** analyzes your code to warn you when you have accidentally published an API key or when one of the dependencies you are using has a vulnerability.
- **Actions/workflows:** a CI/CD tool that allows you to run scripts for every commit, issue, discussion, PR, etc.
- **Releases:** it is possible to publish artifacts/binaries linked to a specific tag.
- **Codespaces:** an online workspace (virtual machine).

Branch protection

The **branch protection system** allows you to define advanced validation rules before a commit can be added to a branch. A few examples:

- Mandatory pull request.
- Minimum number of positive reviews.
- Validated CI workflows.
- Mandatory squash and rebase.
- Push forbidden.

Settings → Branches

Branch protection

Branch name pattern *

main

Applies to 1 branch

main

Protect matching branches

☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☒ **Require approvals**
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.
Required number of approvals before merging: 1 ▾

☒ **Dismiss stale pull request approvals when new commits are pushed**
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☒ **Require review from Code Owners**
Require an approved review in pull requests including files with a designated code owner.

Rules

A more advanced **rules** system allows you to apply similar rules to specific branches/tags or to branches/tags following a pattern, for example: release/*, v*.*.*, etc.

Settings → Rules → Rulesets

Rules

The screenshot displays the configuration page for a ruleset named "Auto-imported tag create protections". The interface is divided into a left sidebar and a main content area.

Left Sidebar:

- General** (selected)
- Access**
 - Collaborators and teams
 - Moderation options
- Code and automation**
 - Branches
 - Tags
 - Rules** (expanded)
 - Rulesets** (selected)
 - Insights
 - Dashboard
 - Actions
 - Models
 - Webhooks
 - Copilot
 - Environments
 - Codespaces
 - Pages
 - Custom properties
- Security and quality**
 - Advanced Security

Main Content Area:

Rulesets / Auto-imported tag create protections Active

Ruleset Name *
Auto-imported tag create protections

Enforcement status
Active

Bypass list + Add bypass
Exempt roles, teams, agents, and apps from this ruleset by adding them to the bypass list.

	Organization admin Role	Always allow	...
	Maintain Role	Always allow	...
	Repository admin Role	Always allow	...

Target tags
Which tags should be matched?

Tag targeting criteria Add target

	libs-v*	
	v*	

Applies to 85 targets including v9.6.1, v9.6.0, v9.5.0, v9.4.0, v9.3.0, v9.2.0, v9.1.0, v9.0.0, v8.4.0, v8.3.0 and others.

Scans: vulnerabilities, secrets and code quality

You can activate **Dependabot** as well as the **secret scanner** (passwords, API keys, etc.) and the **code quality analyzer** to receive alerts/comments when a problem is detected.

"Security and quality" tab

Scans: vulnerabilities, secrets and code quality

Security and quality

Overview

Findings

Code quality

Standard findings

AI findings

Dependabot

Malware

Vulnerabilities 25

Code scanning

Secret scanning

Default

Generic

Reporting

Security policy

Advisories 1

Dependabot alerts

[Give feedback](#)

Dependency files checked 2 days ago

25 Open

331 Closed

☐ Package

☐ Ecosystem

☐ Manifest

☐ Severity

☐ Sort

☐ **Vite: `server.fs.deny` bypassed with queries** High Development

#427 opened last month • Detected in vite (npm) • package-lock.json

☐ **Vite Vulnerable to Arbitrary File Read via Vite Dev Server WebSocket** High Development

#428 opened last month • Detected in vite (npm) • package-lock.json

☐ **basic-ftp has FTP Command Injection via CRLF** High Development

#436 opened last month • Detected in basic-ftp (npm) • package-lock.json

☐ **Race Condition in node-tar Path Reservations via Unicode Ligature Collisions on macOS APFS** High Development

#342 opened last month • Detected in tar (npm) • package-lock.json

☐ **node-tar Vulnerable to Arbitrary File Creation/Overwrite via Hardlink Path Traversal** High Development

#349 opened last month • Detected in tar (npm) • package-lock.json

☐ **node-tar Symlink Path Traversal via Drive-Relative Linkpath** High Development

#394 opened last month • Detected in tar (npm) • package-lock.json

☐ **tar has Hardlink Path Traversal via Drive-Relative Linkpath** High Development

#393 opened last month • Detected in tar (npm) • package-lock.json

Scans: vulnerabilities, secrets and code quality

Security and quality

Overview

Findings

Code quality

Standard findings

AI findings

Dependabot

Malware

Vulnerabilities 25

Code scanning

Secret scanning

Default

Generic

Reporting

Security policy

Advisories 1

Secret scanning alerts

Filter 1

is:open

x

☐

0 Open

1 Closed

Validity

Secret type

Provider

Sort



No secrets found.

Your repository doesn't have any unresolved secrets.

Copyright 1kB SARL-S - contact@1kb.software - 1kb.software

Scans: vulnerabilities, secrets and code quality

Security and quality

- Overview
- Findings
 - Code quality**
 - Standard findings
 - AI findings
 - Dependabot
 - Malware
 - Vulnerabilities 25
 - Code scanning
 - Secret scanning
 - Default
 - Generic
- Reporting
 - Security policy
 - Advisories 1

Standard findings Preview

View findings from a full CodeQL scan of your codebase and improve your code quality scores. To see code quality findings from AI in your most recently changed files go to [AI findings](#).

Maintainability

Fair

1 findings. [Learn how this score is calculated.](#)

Reliability

Fair

6 findings. [Learn how this score is calculated.](#)

Suggestions powered by AI

0 Findings

[Take action](#) on suggestions on recently changed files of your codebase.

Filter 1 is:open

4 rules

Number of findings

Useless conditional 3

Detected by CodeQL · Last seen 1 minute ago on main in [4010084](#)

Reliability Warning JavaScript

Comparison between invertible types 2

Detected by CodeQL · Last seen 1 minute ago on main in [4010084](#)

Reliability Warning JavaScript

Duplicate character in character class 1

Detected by CodeQL · Last seen 1 minute ago on main in [4010084](#)

Reliability Warning JavaScript

Useless assignment to local variable 1

Detected by CodeQL · Last seen 1 minute ago on main in [4010084](#)

Maintainability Warning JavaScript

Copyright 1KB SARL-S - contact@1kb.software - 1kb.software

Actions workflows

GitHub Actions is a **continuous integration** and **continuous delivery** (CI/CD) system that allows you to run scripts or tasks based on a wide range of criteria.

A few examples of use cases:

- Run **automated test** scripts on every commit.
- **Build** the application when a PR is opened to run more extensive tests.
- **Manually trigger the compilation of binary files** (.exe, etc.) upon merging into main/master or when creating a release.
- **Automatically publish** files, binaries, or artifacts to the App Store, Windows Store, etc.

 [Documentation](#)

Actions workflows

Workflows are written in **YAML**. These files must be added to a **.github/workflows** folder at the root of your project.

They operate on a system of **jobs** and **steps**. Each job is a task to execute and can be composed of multiple steps (e.g., installation, running tests, verification, etc.).

Each step consists of one or more **scripts**, which can also be published and reused as an **action**.

Jobs → Steps → Actions

Workflows are executed inside clean **virtual machines** every single time they run.

```
12   jobs:
13     lint:
14       name: Lint and format
15       runs-on: ubuntu-latest
16       timeout-minutes: 10
17
18     steps:
19       - uses: actions/checkout@de0fac2e4500dabe000
20       - name: Install Node.js
21         uses: actions/setup-node@48b55a011bda9f5d6
22         with:
23           node-version: "24"
24           cache: npm
25       - name: Install dependencies
26         run: npm ci
27       - name: Lint and check format
28         run: |
29           npm run lint
30           npm run format
31
```

Actions workflows

GitHub (and other companies) provide ready-made actions for common tasks:

- **actions/checkout**: clones the repository into the virtual machine.
- **actions/setup-node**: installs Node.js.
- **actions/setup-python**: installs Python.
- **actions/cache**: caches specific files to speed up the next execution.

```
12     jobs:
13         lint:
14             name: Lint and format
15             runs-on: ubuntu-latest
16             timeout-minutes: 10
17
18         steps:
19             - uses: actions/checkout@de0fac2e4500dabe000
20             - name: Install Node.js
21               uses: actions/setup-node@48b55a011bda9f5d6
22               with:
23                 node-version: "24"
24                 cache: npm
25             - name: Install dependencies
26               run: npm ci
27             - name: Lint and check format
28               run: |
29                 npm run lint
30                 npm run format
31
```

Actions workflows

An "Actions" tab allows you to view the progress and status of each workflow:

The screenshot displays the GitHub Actions interface. On the left, a sidebar lists workflow runs, including 'Code Quality: CodeQL Setup', 'Add required files for Flathub distribution', 'PR #2172', 'Bump eslint from 10.1.0 to 10.2.1', 'Bump prettier from 3.8.1 to 3.8.3', 'npm_and_yarn in / for eslint - Update #1345167415', and 'npm_and_yarn in / for prettier - Update #13451674...'. The main panel shows the details for the 'Add required files for Flathub distribution #4810' workflow. It includes a 'Summary' tab, a table of workflow runs, and a 'ci.yml' workflow diagram. The workflow diagram shows a sequence of jobs: 'Lint and format' (1m 56s), 'Matrix: Libs - Build and test' (4 jobs completed), 'Matrix: CLI - Build and test' (4 jobs completed), 'Serverless - Build and test' (1m 27s), 'Desktop packaged Windows - Build and test' (24m 26s), and 'Desktop packaged Linux - Build and test' (21m 55s).

2,500+ workflow runs

- ✓ **Code Quality: CodeQL Setup**
CodeQL #1282: by 255kb
- ✓ **Add required files for Flathub distribution**
CI #4810: Pull request #2172 synchronize by 255kb
- ✓ **PR #2172**
CodeQL #1281: by 255kb
- ✗ **Bump eslint from 10.1.0 to 10.2.1**
CI #4809: Pull request #2184 synchronize by dependabot Bot
- ✓ **Bump prettier from 3.8.1 to 3.8.3**
CI #4808: Pull request #2182 synchronize by dependabot Bot
- ✓ **npm_and_yarn in / for eslint - Update #1345167415**
Dependabot Updates #678: by dependabot Bot
- ✓ **npm_and_yarn in / for prettier - Update #13451674...**
Dependabot Updates #677: by dependabot Bot

← CI

✓ **Add required files for Flathub distribution #4810** Re-run all jobs ...

Summary

All jobs

Triggered via	Status	Total duration	Artifacts
pull request yesterday	Success	28m 23s	-
255kb synchronize #2172 feature/1814-Flathub			

ci.yml
on: pull_request

Matrix: Libs - Build and test

- ✓ Lint and format 1m 56s
- ✓ 4 jobs completed
[Show all jobs](#)
- Matrix: CLI - Build and test
- ✓ 4 jobs completed
[Show all jobs](#)
- ✓ Serverless - Build and test 1m 27s
- ✓ Desktop packaged Windows - Build and test 24m 26s
- ✓ Desktop packaged Linux - Build and test 21m 55s

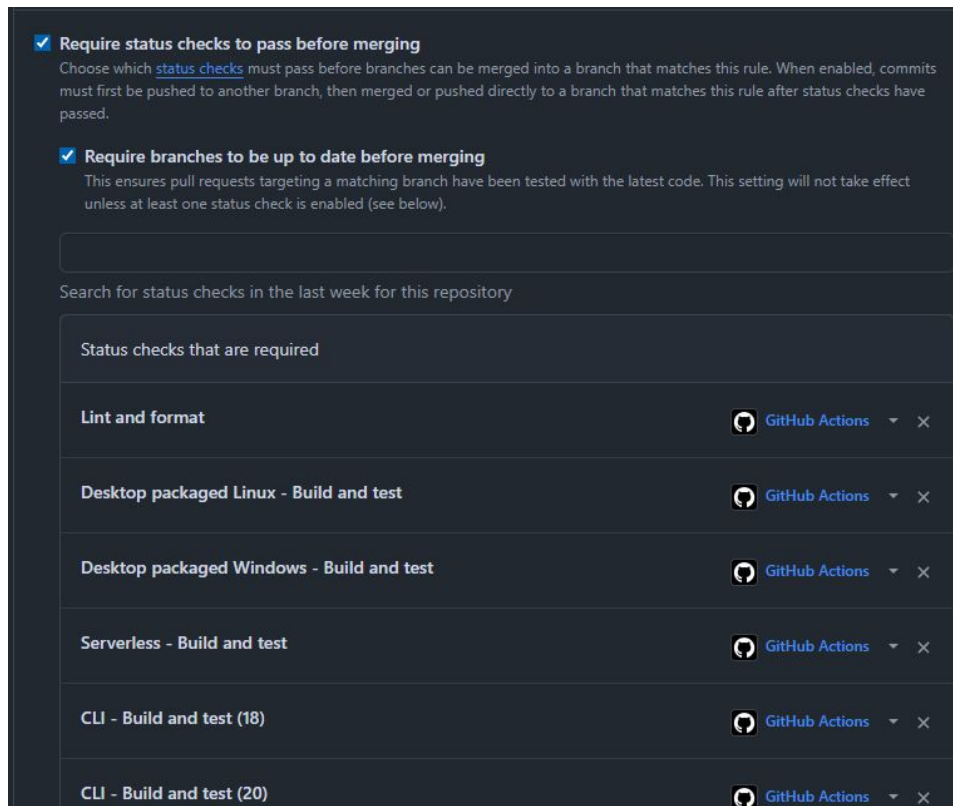
Run details

Usage

Workflow file

Actions workflows

The (positive) result of the workflows can be added as a **condition for merging PRs** using branch protection rules.



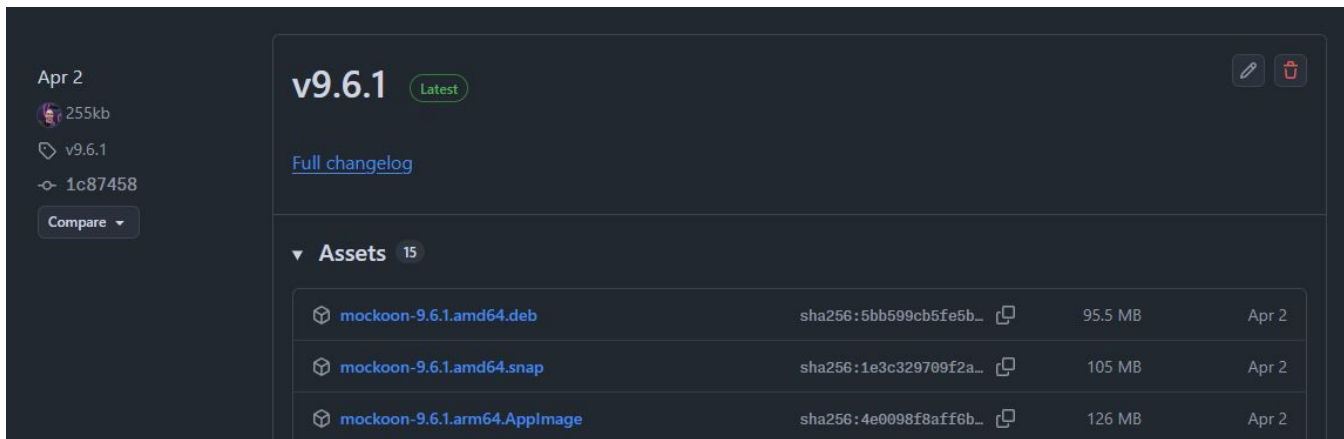
The screenshot displays the 'Require status checks to pass before merging' configuration in a GitHub repository. It shows two checked options: 'Require status checks to pass before merging' and 'Require branches to be up to date before merging'. Below these, a search bar is present, followed by a table of required status checks. The table lists six checks, all using GitHub Actions, with expand and close icons for each.

Status checks that are required		
Lint and format	GitHub Actions	✕
Desktop packaged Linux - Build and test	GitHub Actions	✕
Desktop packaged Windows - Build and test	GitHub Actions	✕
Serverless - Build and test	GitHub Actions	✕
CLI - Build and test (18)	GitHub Actions	✕
CLI - Build and test (20)	GitHub Actions	✕

Releases

GitHub offers a **release** system linked to a published **tag** on GitHub. Releases can contain a Markdown **description** as well as **files** (often binaries: .exe, .dmg, etc.).

Releases are frequently used by third-party distribution tools (such as Homebrew, Chocolatey, etc.).



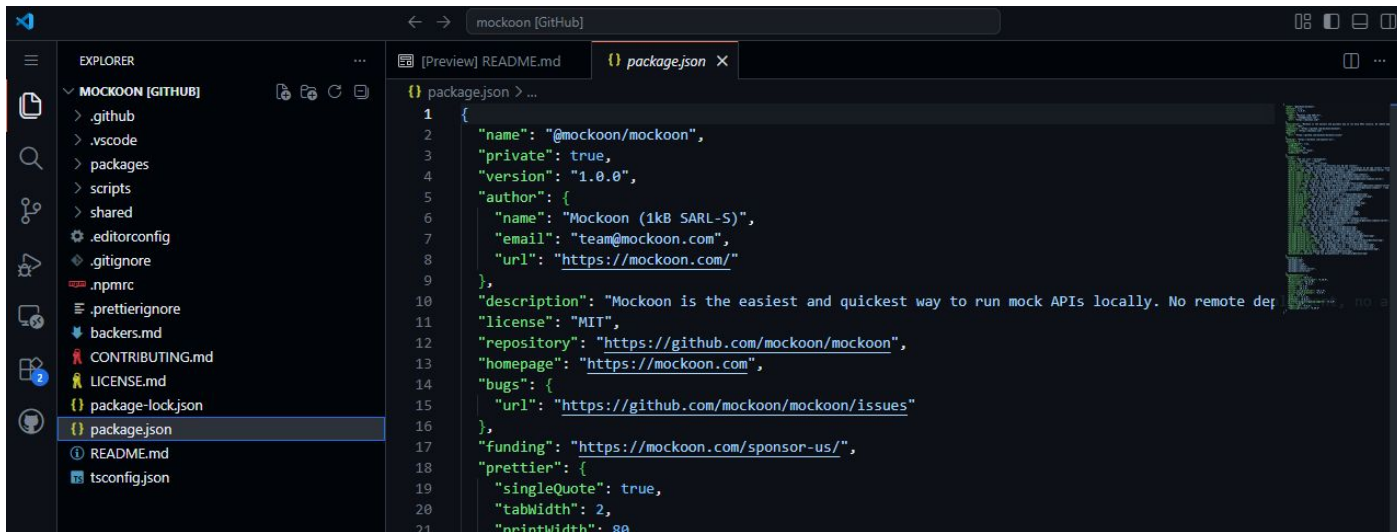
The screenshot shows a GitHub release page for version **v9.6.1**, which is marked as the **Latest** release. The page includes a sidebar with the release date (Apr 2), a download icon, the version number (v9.6.1), a commit hash (1c87458), and a 'Compare' button. The main content area features a 'Full changelog' link and a section titled 'Assets' with 15 items. Three assets are visible in the table below:

Asset Name	SHA256 Hash	Size	Date
mockoon-9.6.1.amd64.deb	sha256:5bb599cb5fe5b...	95.5 MB	Apr 2
mockoon-9.6.1.amd64.snap	sha256:1e3c329709f2a...	105 MB	Apr 2
mockoon-9.6.1.arm64.AppImage	sha256:4e0098f8aff6b...	126 MB	Apr 2

Codespaces

It is possible to launch an **online VS Code instance** simply by pressing the “.” key on any repository.

Codespaces run on a virtual machine and offer a free tier for personal accounts, calculated in hours per month (120 hours).



GitHub tips

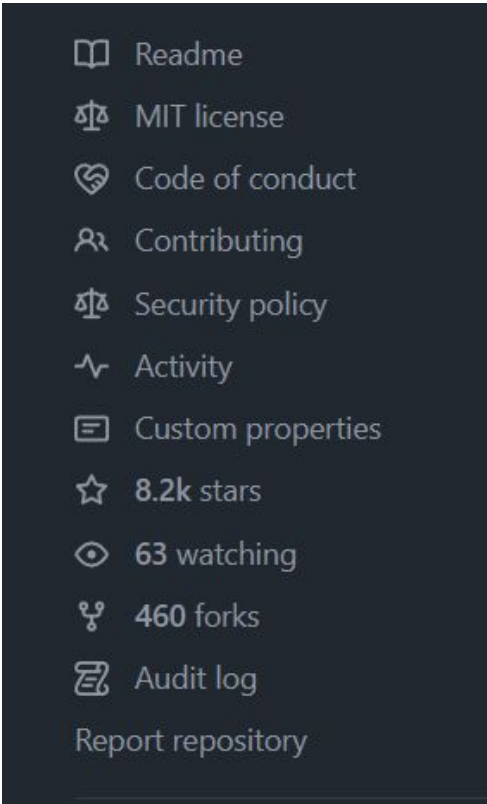
Markdown

GitHub supports [markdown](#) with a few additions ([GitHub flavored Markdown](#)):
tables, task lists, special links (@people, #issue_number, #pull_request_number),
file uploads (simple Ctrl+V), etc.

Markdown is supported almost everywhere.

Readme, license, security, contributing, code of conduct...

The creation and publication of certain files (README.md, CONTRIBUTING.md, LICENSE.md, etc.) automatically triggers the display of specific sections, particularly on the repository's homepage:

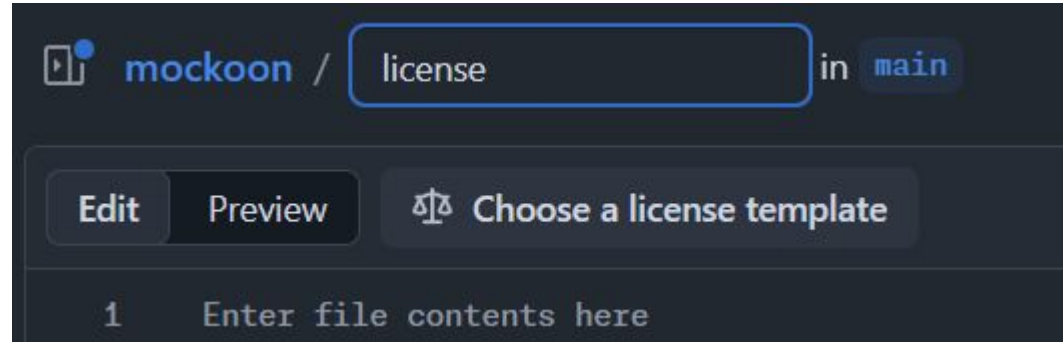
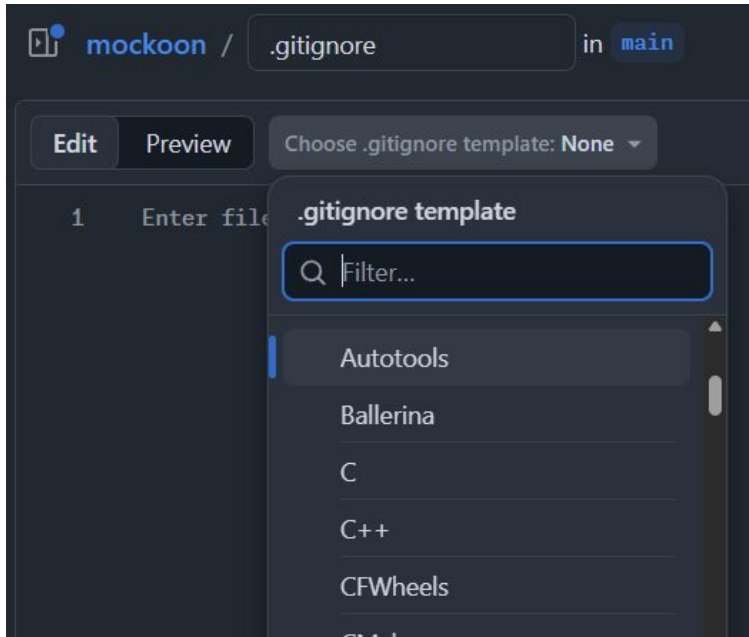


A dark-themed sidebar menu from a GitHub repository homepage. It lists various sections with corresponding icons: a book icon for 'Readme', a document with a checkmark for 'MIT license', a heart in a circle for 'Code of conduct', a person with a checkmark for 'Contributing', a document with a checkmark for 'Security policy', a pulse line for 'Activity', a list icon for 'Custom properties', a star for '8.2k stars', an eye for '63 watching', a fork icon for '460 forks', a document with a checkmark for 'Audit log', and a report icon for 'Report repository'.

- Readme
- MIT license
- Code of conduct
- Contributing
- Security policy
- Activity
- Custom properties
- 8.2k stars
- 63 watching
- 460 forks
- Audit log
- Report repository

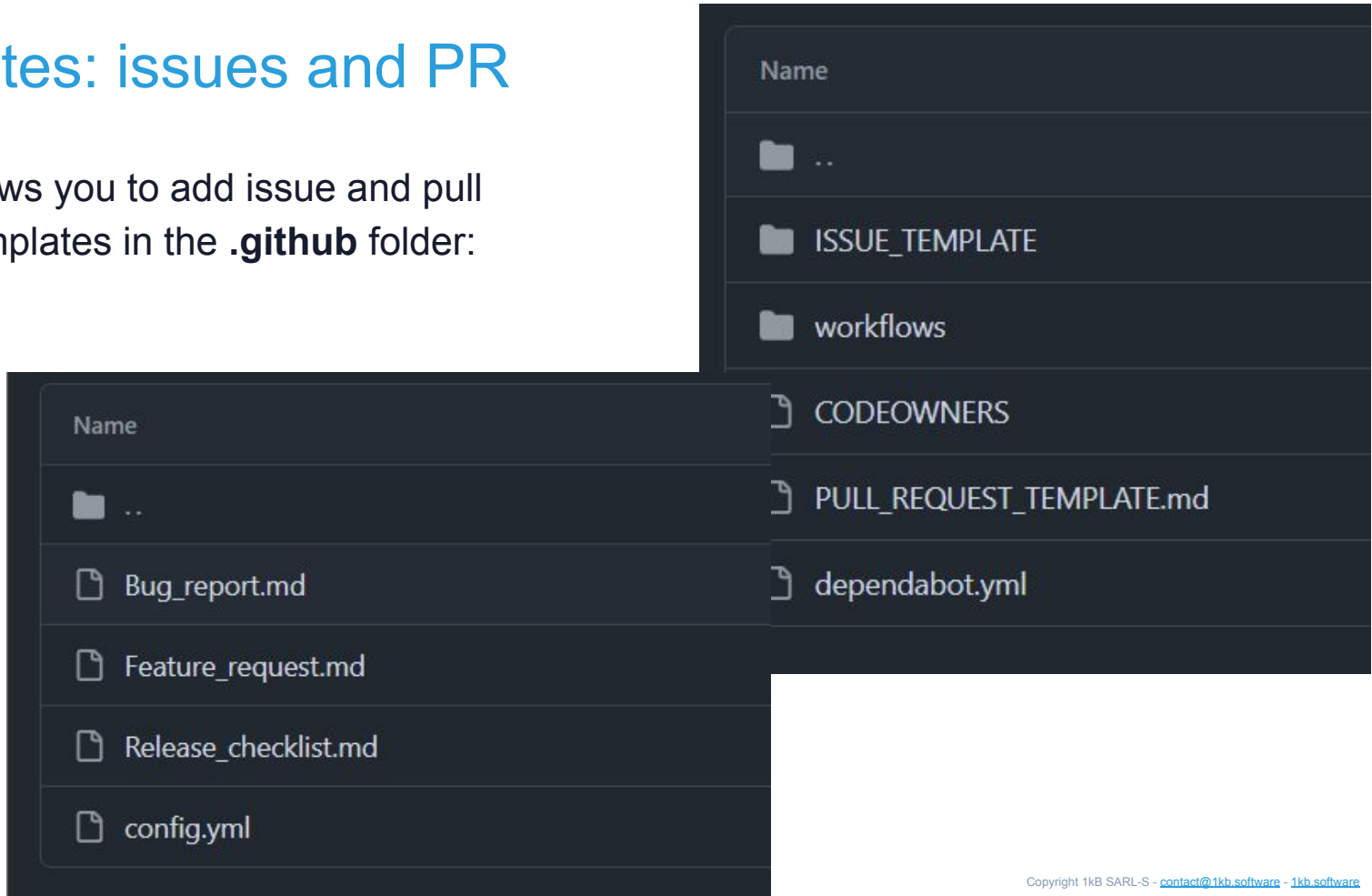
Readme, license, security, contributing, code of conduct...

When you create certain files online, GitHub will offer you templates, notably for .gitignore and LICENSE.md files:



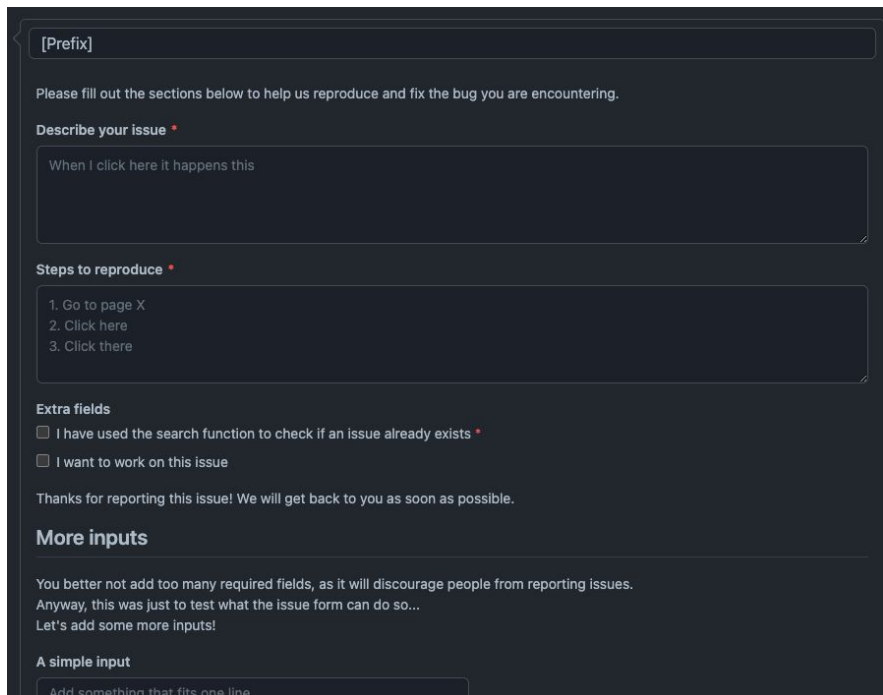
Templates: issues and PR

GitHub allows you to add issue and pull request templates in the **.github** folder:



Templates: issues and PR

The creation of issues or PRs will then suggest a template to follow in Markdown format, or for issues, even in the form of a structured form:



[Prefix]

Please fill out the sections below to help us reproduce and fix the bug you are encountering.

Describe your issue *

When I click here it happens this

Steps to reproduce *

1. Go to page X
2. Click here
3. Click there

Extra fields

☐ I have used the search function to check if an issue already exists *

☐ I want to work on this issue

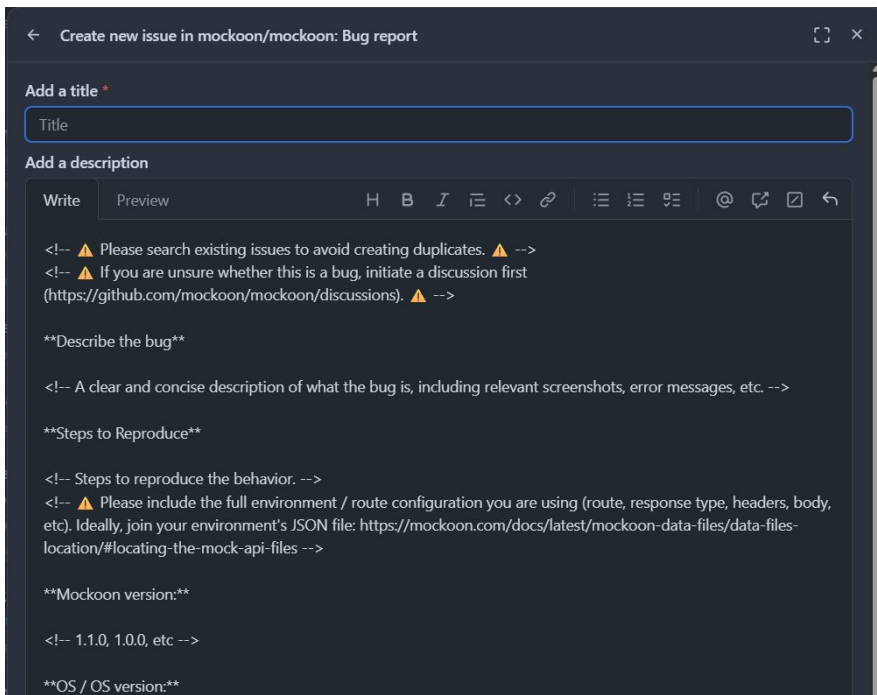
Thanks for reporting this issue! We will get back to you as soon as possible.

More inputs

You better not add too many required fields, as it will discourage people from reporting issues. Anyway, this was just to test what the issue form can do so... Let's add some more inputs!

A simple input

Add something that fits one line



← Create new issue in mockoon/mockoon: Bug report

Add a title *

Title

Add a description

Write Preview H B I

<!-- 🚩 Please search existing issues to avoid creating duplicates. 🚩 -->
<!-- 🚩 If you are unsure whether this is a bug, initiate a discussion first (<https://github.com/mockoon/mockoon/discussions>). 🚩 -->

****Describe the bug****

<!-- A clear and concise description of what the bug is, including relevant screenshots, error messages, etc. -->

****Steps to Reproduce****

<!-- Steps to reproduce the behavior. -->
<!-- 🚩 Please include the full environment / route configuration you are using (route, response type, headers, body, etc). Ideally, join your environment's JSON file: <https://mockoon.com/docs/latest/mockoon-data-files/data-files-location/#locating-the-mock-api-files> -->

****Mockoon version:****

<!-- 1.1.0, 1.0.0, etc -->

****OS / OS version:****